Homework 3 — Solutions

CE 512 Finite Element Method

2026 Spring

Muhammet Yağcıoğlu

Student No: 290204042

March 16, 2026

GitHub: `https://github.com/adzetto/ce512-fem-lab`

İzmir Institute of Technology
Department of Civil Engineering

Contents

1 Problem Statement **3**

2 Potential Energy Functional **3**

3 Part (a): One-Term Approximation, $u(x) = a_2x^2$ **3**
3.1 Derivation . . . 3
3.2 Displacement and Stress Fields . . . 4

4 Part (b): Two-Term Approximation, $u(x) = a_2x^2 + a_3x^3$ **4**
4.1 Derivation . . . 4
4.2 Solution of the 2×2 System . . . 6
4.3 Verification . . . 7
4.4 Displacement and Stress Fields . . . 7

5 Exact Solution **8**

6 Part (c): Displacement Plots **8**

7 Part (d): Stress Plots **9**

8 Discussion **10**
8.1 Incomplete Trial Functions and Convergence . . . 10
8.2 Overly Stiff Approximation . . . 10
8.3 Displacement Accuracy vs. Stress Accuracy . . . 11
8.4 Physical Interpretation of the Plots . . . 11
8.5 Effect of Restoring the Linear Term . . . 11

9 Problem 2: Bilinear Rectangular Finite Element **12**
9.1 Potential Energy and Rayleigh-Ritz Equations . . . 12
9.2 Derivation of the Shape Functions . . . 12
9.3 Displacement Interpolation . . . 14
9.4 Strain–Displacement Relation and Element Stiffness . . . 15
9.5 Natural (Isoparametric) Coordinates . . . 15
9.6 Discussion of Problem 2 . . . 17

10 Numerical Validation **17**
10.1 OpenSees Validation — Question 1 . . . 17
10.2 FemLab Validation — Question 1 . . . 18
10.3 FemLab Validation — Question 2 . . . 18

11 Convergence Study: n-Term Polynomial **19**
11.1 Function Space and Admissibility . . . 19
11.2 General n-Term Trial Function . . . 19
11.3 Total Potential Energy . . . 19
11.4 Stationarity and the Rayleigh-Ritz System . . . 21
11.5 Explicit Matrix Entries . . . 21
11.6 Displacement and Stress from the n-Term Solution . . . 22
11.7 Error Measures . . . 23
11.8 Numerical Implementation . . . 24
11.9 Large-Scale Solve: n up to 10 000 . . . 26

12 Conclusion **28**

A Source Code **29**
A.1 Julia — Rayleigh-Ritz Convergence Study 29
A.2 Julia — Large-Scale Scaling Study (n up to 10 000) 32
A.3 Python — OpenSees Validation 34

1 Problem Statement

A uniform elastic bar of length L, cross-sectional area A, and Young's modulus E is fixed at the left end ($x = 0$) and free at the right end ($x = L$). Two concentrated loads, each of magnitude P, act in the positive x-direction: one at the mid-length $x = L/2$ and one at the right end $x = L$ (Figure 1).

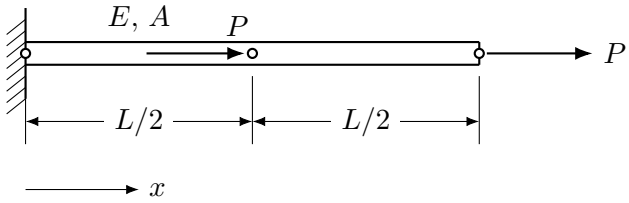

Figure 1: 1-D bar element: fixed at left end, loads P at $x = L/2$ and $x = L$.

Required:

(a) Find a_2 for $u(x) = a_2 x^2$.

(b) Find a_2, a_3 for $u(x) = a_2 x^2 + a_3 x^3$.

(c) Plot the displacement fields from (a) and (b) on a single graph.

(d) Plot the stress variation for (a) and (b) on a single graph.

2 Potential Energy Functional

Let $\varepsilon_x = u_{,x}$, $\sigma_x = E\varepsilon_x$. Total potential energy:

$$\Pi_p = U + \Omega = \frac{EA}{2}\int_0^L u_{,x}^2\,dx - P\,u\big(\tfrac{L}{2}\big) - P\,u(L) \tag{1}$$

Essential BC: $u(0) = 0$. Both trial functions satisfy this automatically.
Let $u(x) = \sum_j a_j\,\phi_j(x)$ with $\phi_j(x) = x^j$. Stationarity $\partial\Pi_p/\partial a_i = 0$ gives

$$[\mathbf{K}]\{\mathbf{a}\} = \{\mathbf{F}\} \tag{2}$$

$$K_{ij} = EA\int_0^L \phi_i'(x)\,\phi_j'(x)\,dx, \qquad F_i = P\,\phi_i\big(\tfrac{L}{2}\big) + P\,\phi_i(L) \tag{3}$$

3 Part (a): One-Term Approximation, $u(x) = a_2 x^2$

3.1 Derivation

Let $\phi_2(x) = x^2$, $\phi_2'(x) = 2x$, so $u(x) = a_2\,x^2$, $u_{,x} = 2a_2\,x$.

Stiffness coefficient K_{22}.

$$\begin{aligned} K_{22} &= EA\int_0^L \phi_2'(x)\;\phi_2'(x)\,dx = EA\int_0^L (2x)(2x)\,dx \\ &= 4\,EA\int_0^L x^2\,dx = 4\,EA\left[\frac{x^3}{3}\right]_0^L \\ &= 4\,EA\cdot\frac{L^3}{3} = \frac{4\,EAL^3}{3} \end{aligned} \tag{4}$$

Force coefficient F_2.

$$F_2 = P\,\phi_2\left(\frac{L}{2}\right) + P\,\phi_2(L) = P\left(\frac{L}{2}\right)^2 + P\,L^2$$
$$= \frac{PL^2}{4} + PL^2 = \frac{PL^2}{4} + \frac{4PL^2}{4} = \frac{5PL^2}{4} \tag{5}$$

Strain energy (via K_{22}).

$$U = \tfrac{1}{2}\,a_2\,K_{22}\,a_2 = \tfrac{1}{2}\cdot\frac{4EAL^3}{3}\,a_2^2 = \frac{2EAL^3}{3}\,a_2^2 \tag{6}$$

Total potential energy.

$$\Pi_p = U + \Omega = \frac{2EAL^3}{3}\,a_2^2 - \frac{5PL^2}{4}\,a_2 \tag{7}$$

Stationarity. $\partial\Pi_p/\partial a_2 = 0$:

$$\frac{\partial\Pi_p}{\partial a_2} = \frac{4EAL^3}{3}\,a_2 - \frac{5PL^2}{4} = 0 \qquad \Longrightarrow \qquad K_{22}\,a_2 = F_2$$

Solving for a_2.

$$a_2 = \frac{F_2}{K_{22}} = \frac{\dfrac{5PL^2}{4}}{\dfrac{4EAL^3}{3}} = \frac{5PL^2}{4}\cdot\frac{3}{4EAL^3}$$
$$= \frac{5\cdot 3\;P\,L^2}{4\cdot 4\;EA\,L^3} = \frac{15\,P}{16\,EA\,L}$$

$$\boxed{a_2 = \frac{15\,P}{16\,EAL}} \tag{8}$$

3.2 Displacement and Stress Fields

$$\boxed{u_a(x) = \frac{15P}{16\,EAL}\,x^2} \tag{9}$$

$$\sigma_a(x) = E\,u_{a,x} = E\cdot\frac{15P\cdot 2x}{16\,EAL} = \frac{15\,P\,x}{8\,A\,L} \tag{10}$$

Key values:

$$u_a\left(\frac{L}{2}\right) = \frac{15PL}{64\,EA}, \qquad u_a(L) = \frac{15PL}{16\,EA}, \qquad \sigma_a(0) = 0, \qquad \sigma_a(L) = \frac{15P}{8A}.$$

4 Part (b): Two-Term Approximation, $u(x) = a_2x^2 + a_3x^3$

4.1 Derivation

Let $\phi_2 = x^2$, $\phi_3 = x^3$, so $\phi_2' = 2x$, $\phi_3' = 3x^2$, $u = a_2x^2 + a_3x^3$, $u_{,x} = 2a_2x + 3a_3x^2$.

Stiffness coefficients K_{ij}. K_{22}:

$$K_{22} = EA\int_0^L \phi_2'\,\phi_2'\,dx = EA\int_0^L (2x)(2x)\,dx = 4\,EA\int_0^L x^2\,dx$$
$$= 4\,EA\cdot\frac{L^3}{3} = \frac{4\,EAL^3}{3}$$

$K_{23} = K_{32}$ (symmetry):

$$K_{23} = EA\int_0^L \phi_2'\,\phi_3'\,dx = EA\int_0^L (2x)(3x^2)\,dx = 6\,EA\int_0^L x^3\,dx$$
$$= 6\,EA\cdot\frac{L^4}{4} = \frac{6\,EAL^4}{4} = \frac{3\,EAL^4}{2}$$

K_{33}:

$$K_{33} = EA\int_0^L \phi_3'\,\phi_3'\,dx = EA\int_0^L (3x^2)(3x^2)\,dx = 9\,EA\int_0^L x^4\,dx$$
$$= 9\,EA\cdot\frac{L^5}{5} = \frac{9\,EAL^5}{5}$$

Force coefficients F_i. F_2:

$$F_2 = P\,\phi_2\left(\frac{L}{2}\right) + P\,\phi_2(L) = P\left(\frac{L}{2}\right)^2 + P\,L^2 = \frac{PL^2}{4} + PL^2 = \frac{5PL^2}{4}$$

F_3:

$$F_3 = P\,\phi_3\left(\frac{L}{2}\right) + P\,\phi_3(L) = P\left(\frac{L}{2}\right)^3 + P\,L^3 = \frac{PL^3}{8} + PL^3 = \frac{PL^3 + 8PL^3}{8} = \frac{9PL^3}{8}$$

Strain energy (check).

$$u_{,x}^2 = (2a_2x + 3a_3x^2)^2 = 4a_2^2\,x^2 + 12\,a_2a_3\,x^3 + 9a_3^2\,x^4$$

$$U = \frac{EA}{2}\int_0^L \left(4a_2^2\,x^2 + 12\,a_2a_3\,x^3 + 9a_3^2\,x^4\right)dx$$
$$= \frac{EA}{2}\left[4a_2^2\cdot\frac{L^3}{3} + 12\,a_2a_3\cdot\frac{L^4}{4} + 9a_3^2\cdot\frac{L^5}{5}\right]$$
$$= EA\left[\frac{2a_2^2L^3}{3} + \frac{3\,a_2a_3L^4}{2} + \frac{9a_3^2L^5}{10}\right] = \frac{1}{2}\,a_i\,K_{ij}\,a_j \quad \checkmark$$

Potential of external forces.

$$u\left(\frac{L}{2}\right) = a_2\left(\frac{L}{2}\right)^2 + a_3\left(\frac{L}{2}\right)^3 = \frac{a_2L^2}{4} + \frac{a_3L^3}{8}$$
$$u(L) = a_2L^2 + a_3L^3$$

$$\Omega = -P\,u\left(\frac{L}{2}\right) - P\,u(L)$$
$$= -P\left[\frac{a_2L^2}{4} + \frac{a_3L^3}{8} + a_2L^2 + a_3L^3\right]$$
$$= -P\left[\left(\frac{1}{4}+1\right)a_2L^2 + \left(\frac{1}{8}+1\right)a_3L^3\right] = -P\left[\frac{5a_2L^2}{4} + \frac{9a_3L^3}{8}\right]$$

Total potential energy.

$$\Pi_p = EA\left[\frac{2L^3}{3}\,a_2^2 + \frac{3L^4}{2}\,a_2 a_3 + \frac{9L^5}{10}\,a_3^2\right] - P\left[\frac{5L^2}{4}\,a_2 + \frac{9L^3}{8}\,a_3\right] \tag{11}$$

Stationarity. $\partial\Pi_p/\partial a_i = 0$:

$$\frac{\partial\Pi_p}{\partial a_2} = EA\left[\frac{4L^3}{3}\,a_2 + \frac{3L^4}{2}\,a_3\right] - \frac{5PL^2}{4} = 0 \tag{12}$$

$$\frac{\partial\Pi_p}{\partial a_3} = EA\left[\frac{3L^4}{2}\,a_2 + \frac{9L^5}{5}\,a_3\right] - \frac{9PL^3}{8} = 0 \tag{13}$$

In matrix form $[\mathbf{K}]\{\mathbf{a}\} = \{\mathbf{F}\}$:

$$EA\begin{bmatrix} \dfrac{4L^3}{3} & \dfrac{3L^4}{2} \\ \dfrac{3L^4}{2} & \dfrac{9L^5}{5} \end{bmatrix}\begin{Bmatrix} a_2 \\ a_3 \end{Bmatrix} = P\begin{Bmatrix} \dfrac{5L^2}{4} \\ \dfrac{9L^3}{8} \end{Bmatrix} \tag{14}$$

4.2 Solution of the 2×2 System

Determinant. $\det[\mathbf{K}] = K_{22}K_{33} - K_{23}^2$:

$$K_{22}\,K_{33} = \frac{4EAL^3}{3}\cdot\frac{9EAL^5}{5} = \frac{36\,(EA)^2L^8}{15} = \frac{12\,(EA)^2L^8}{5}$$

$$K_{23}^2 = \left(\frac{3EAL^4}{2}\right)^2 = \frac{9\,(EA)^2L^8}{4}$$

$$\det[\mathbf{K}] = \frac{12\,(EA)^2L^8}{5} - \frac{9\,(EA)^2L^8}{4} = (EA)^2L^8\left(\frac{12}{5} - \frac{9}{4}\right)$$

$$= (EA)^2L^8\cdot\frac{48-45}{20} = \frac{3\,(EA)^2L^8}{20}$$

Inverse. $[\mathbf{K}]^{-1} = \frac{1}{\det[\mathbf{K}]}\begin{bmatrix} K_{33} & -K_{23} \\ -K_{32} & K_{22} \end{bmatrix}$:

$$[\mathbf{K}]^{-1} = \frac{20}{3\,(EA)^2L^8}\begin{bmatrix} \dfrac{9EAL^5}{5} & -\dfrac{3EAL^4}{2} \\ -\dfrac{3EAL^4}{2} & \dfrac{4EAL^3}{3} \end{bmatrix}$$

$$= \frac{1}{EA}\begin{bmatrix} \dfrac{20\cdot 9}{3\cdot 5\,L^3} & \dfrac{-20\cdot 3}{3\cdot 2\,L^4} \\ \dfrac{-20\cdot 3}{3\cdot 2\,L^4} & \dfrac{20\cdot 4}{3\cdot 3\,L^5} \end{bmatrix} = \frac{1}{EA}\begin{bmatrix} \dfrac{12}{L^3} & -\dfrac{10}{L^4} \\ -\dfrac{10}{L^4} & \dfrac{80}{9L^5} \end{bmatrix}$$

Computing a_2.

$$\begin{aligned}
a_2 &= (K^{-1})_{22}\,F_2 + (K^{-1})_{23}\,F_3 \\
&= \frac{1}{EA}\left[\frac{12}{L^3}\cdot\frac{5PL^2}{4} - \frac{10}{L^4}\cdot\frac{9PL^3}{8}\right] \\
&= \frac{P}{EA}\left[\frac{60}{4L} - \frac{90}{8L}\right] = \frac{P}{EA}\left[\frac{15}{L} - \frac{45}{4L}\right] \\
&= \frac{P}{EA}\cdot\frac{60-45}{4L} = \frac{15\,P}{4\,EA\,L}
\end{aligned}$$

Computing a_3.

$$\begin{aligned}
a_3 &= (K^{-1})_{32}\,F_2 + (K^{-1})_{33}\,F_3 \\
&= \frac{1}{EA}\left[-\frac{10}{L^4}\cdot\frac{5PL^2}{4} + \frac{80}{9L^5}\cdot\frac{9PL^3}{8}\right] \\
&= \frac{P}{EA}\left[-\frac{50}{4L^2} + \frac{720}{72\,L^2}\right] = \frac{P}{EA}\left[-\frac{50}{4L^2} + \frac{10}{L^2}\right] \\
&= \frac{P}{EA}\cdot\frac{-50+40}{4L^2} = -\frac{10\,P}{4\,EA\,L^2} = -\frac{5\,P}{2\,EA\,L^2}
\end{aligned}$$

$$\boxed{a_2 = \frac{15\,P}{4\,EAL}, \qquad a_3 = -\frac{5\,P}{2\,EAL^2}} \tag{15}$$

4.3 Verification

Substituting back into Equation (12):

$$EA\left[\frac{4L^3}{3}\cdot\frac{15P}{4EAL} + \frac{3L^4}{2}\cdot\frac{-5P}{2EAL^2}\right] = 5PL^2 - \frac{15PL^2}{4} = \frac{5PL^2}{4}\ \checkmark$$

Substituting back into Equation (13):

$$EA\left[\frac{3L^4}{2}\cdot\frac{15P}{4EAL} + \frac{9L^5}{5}\cdot\frac{-5P}{2EAL^2}\right] = \frac{45PL^3}{8} - \frac{9PL^3}{2} = \frac{9PL^3}{8}\ \checkmark$$

4.4 Displacement and Stress Fields

$$\boxed{u_b(x) = \frac{15\,P}{4\,EAL}\,x^2 - \frac{5\,P}{2\,EAL^2}\,x^3 = \frac{5\,P\,x^2}{4\,EAL}\left(3 - \frac{2x}{L}\right)} \tag{16}$$

The stress:

$$\sigma_b(x) = E\,u_{b,x} = E\left(\frac{30\,P\,x}{4\,EAL} - \frac{15\,P\,x^2}{2\,EAL^2}\right) = \frac{15\,P\,x}{2\,A\,L}\left(1 - \frac{x}{L}\right) \tag{17}$$

$$\boxed{\sigma_b(x) = \frac{15\,P\,x\,(L-x)}{2\,A\,L^2}} \tag{18}$$

Key values:

$$u_b\left(\frac{L}{2}\right) = \frac{5PL}{8\,EA}, \qquad u_b(L) = \frac{5PL}{4\,EA}, \qquad \sigma_b(0) = 0, \qquad \sigma_b\left(\frac{L}{2}\right) = \frac{15P}{8A}, \qquad \sigma_b(L) = 0.$$

5 Exact Solution

Internal force. Cut at $0 < x < L/2$: $N(x) = 2P$. Cut at $L/2 < x < L$: $N(x) = P$.

$$N(x) = \begin{cases} 2P & 0 \le x < L/2 \\ P & L/2 < x \le L \end{cases}$$

Displacement. $\varepsilon(x) = N(x)/(EA)$, integrate from $u(0) = 0$. For $0 \le x \le L/2$:

$$u(x) = \int_0^x \frac{N(\xi)}{EA}\, d\xi = \int_0^x \frac{2P}{EA}\, d\xi = \frac{2P}{EA}\, x$$

At $x = L/2$: $u(L/2) = \dfrac{2P}{EA} \cdot \dfrac{L}{2} = \dfrac{PL}{EA}$.
For $L/2 \le x \le L$:

$$\begin{aligned} u(x) &= u\left(\frac{L}{2}\right) + \int_{L/2}^x \frac{N(\xi)}{EA}\, d\xi = \frac{PL}{EA} + \int_{L/2}^x \frac{P}{EA}\, d\xi \\ &= \frac{PL}{EA} + \frac{P}{EA}\left(x - \frac{L}{2}\right) = \frac{PL}{EA} + \frac{Px}{EA} - \frac{PL}{2EA} \\ &= \frac{P}{EA}\left(\frac{L}{2} + x\right) \end{aligned}$$

$$\boxed{u_{\text{exact}}(x) = \begin{cases} \dfrac{2Px}{EA} & 0 \le x \le L/2 \\ \dfrac{P(L/2 + x)}{EA} & L/2 \le x \le L \end{cases}} \tag{19}$$

At $x = L$: $u(L) = \dfrac{P}{EA}\left(\dfrac{L}{2} + L\right) = \dfrac{3PL}{2EA}$.

Exact stress.

$$\sigma_{\text{exact}}(x) = \frac{N(x)}{A} = \begin{cases} \dfrac{2P}{A} & 0 \le x < L/2 \\ \dfrac{P}{A} & L/2 < x \le L \end{cases} \tag{20}$$

Key exact values.

$$u_{\text{exact}}\left(\frac{L}{2}\right) = \frac{PL}{EA}, \qquad u_{\text{exact}}(L) = \frac{3PL}{2\,EA}, \qquad \sigma_{\text{exact}}(0^+) = \frac{2P}{A}, \qquad \sigma_{\text{exact}}(L) = \frac{P}{A}.$$

6 Part (c): Displacement Plots

Figure 2 compares the two Rayleigh-Ritz approximations and the exact displacement field. All quantities are non-dimensionalised using $\bar{x} = x/L$ and $\bar{u} = u \cdot EA/(PL)$.

$$\bar{u}_a(\bar{x}) = \frac{15}{16}\,\bar{x}^2 \tag{21}$$

$$\bar{u}_b(\bar{x}) = \frac{15}{4}\,\bar{x}^2 - \frac{5}{2}\,\bar{x}^3 \tag{22}$$

$$\bar{u}_{\text{exact}}(\bar{x}) = \begin{cases} 2\bar{x} & 0 \le \bar{x} \le 0.5 \\ 0.5 + \bar{x} & 0.5 \le \bar{x} \le 1 \end{cases} \tag{23}$$

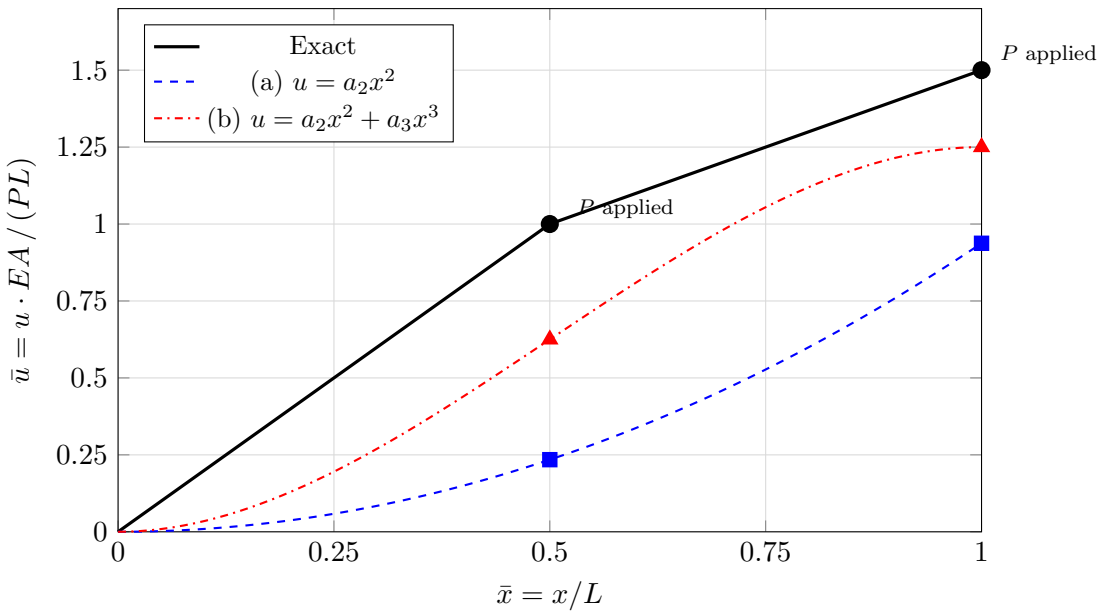

Figure 2: Non-dimensional displacement fields: exact solution vs. one-term and two-term Rayleigh-Ritz approximations.

$\bar{x}$	$\bar{u}_{\text{exact}}$	$\bar{u}_a$	Error (a)	$\bar{u}_b$	Error (b)
0	0	0	—	0	—
0.5	1.000	0.234	76.6%	0.625	37.5%
1.0	1.500	0.938	37.5%	1.250	16.7%

Table 1: Displacement comparison at key locations. Error $= 1 - \bar{u}_{\text{approx}}/\bar{u}_{\text{exact}}$.

7 Part (d): Stress Plots

The non-dimensional stress $\bar{\sigma} = \sigma \cdot A/P$ for each case:

$$\bar{\sigma}_a(\bar{x}) = \frac{15}{8}\,\bar{x} \tag{24}$$

$$\bar{\sigma}_b(\bar{x}) = \frac{15}{2}\,\bar{x}\,(1-\bar{x}) \tag{25}$$

$$\bar{\sigma}_{\text{exact}}(\bar{x}) = \begin{cases} 2 & 0 \le \bar{x} < 0.5 \\ 1 & 0.5 < \bar{x} \le 1 \end{cases} \tag{26}$$

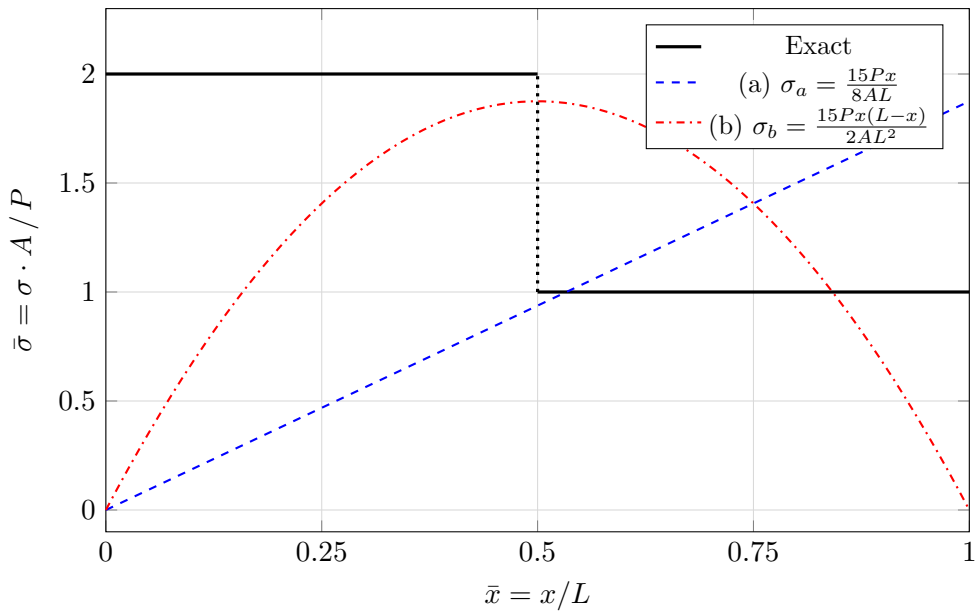

Figure 3: Non-dimensional stress fields: exact solution vs. one-term and two-term Rayleigh-Ritz approximations.

$\bar{x}$	$\bar{\sigma}_{\text{exact}}$	$\bar{\sigma}_a$	$\bar{\sigma}_b$
0	2.000	0	0
0.25	2.000	0.469	1.406
0.50^-	2.000	0.938	1.875
0.50^+	1.000	0.938	1.875
0.75	1.000	1.406	1.406
1.00	1.000	1.875	0

Table 2: Stress comparison at key locations.

8 Discussion

8.1 Incomplete Trial Functions and Convergence

Both trial functions in this problem **omit the linear term** $a_1 x$. As discussed in Cook, Section 3.6, completeness of the polynomial basis requires that no lowest-order admissible terms be omitted. The term $a_1 x$ represents the *constant-strain capability*: it is the only term for which $u_{,x}$ is a nonzero constant. Without it, the trial function is *incomplete*, and convergence to the exact solution is not guaranteed even as more higher-order terms are added.

Concretely, for any trial function of the form $u(x) = a_2 x^2 + a_3 x^3 + \cdots + a_n x^n$, the strain is $\varepsilon(x) = u_{,x} = 2a_2 x + 3a_3 x^2 + \cdots$, which **always vanishes at** $x = 0$. This contradicts the exact solution, where $\sigma(0) = 2P/A$ is the *maximum* stress in the bar.

8.2 Overly Stiff Approximation

The Rayleigh-Ritz method with displacement-based trial functions yields solutions that are *either exact or too stiff* (Cook, Section 3.6). The approximate displacement field imposes

kinematic constraints that prevent the structure from deforming freely, creating an effectively stiffer substitute structure. Our results confirm this:

- **Part (a):** $u_a(L) = \frac{15PL}{16EA} \approx 0.938\,\frac{PL}{EA}$, while $u_{\text{exact}}(L) = \frac{3PL}{2EA} = 1.5\,\frac{PL}{EA}$. The displacement is **underestimated by 37.5%**.
- **Part (b):** $u_b(L) = \frac{5PL}{4EA} = 1.25\,\frac{PL}{EA}$, still **underestimated by 16.7%**.

Adding the a_3x^3 term provides one more degree of freedom, improving the result. However, neither approximation can match the exact piecewise-linear displacement field because the basis lacks the a_1x term.

8.3 Displacement Accuracy vs. Stress Accuracy

Cook, Section 3.5 notes that *approximate displacements are more accurate than approximate stresses*, because stresses are computed from *derivatives* of the displacement field, which amplify discrepancies. Our results illustrate this dramatically:

- **Displacement:** Part (b) is within 16.7% of the exact value at $x = L$.
- **Stress:** At $x = 0$, both approximations give $\sigma = 0$, while the exact stress is $2P/A$ — an error of **100%**. At $x = L$, part (a) gives $\sigma_a(L) = 15P/(8A) = 1.875\,P/A$, **overestimating** the exact value P/A by 87.5%, while part (b) gives $\sigma_b(L) = 0$, **underestimating** by 100%.

This confirms that approximate stresses can be too high in some regions and too low in others, even when the displacements are uniformly underestimated.

8.4 Physical Interpretation of the Plots

From Figure 2:

- The exact displacement is piecewise linear with a slope change at $x = L/2$ (where the concentrated load introduces a discontinuity in the internal force).
- Both polynomial approximations are smooth (no slope discontinuity), so they cannot capture the kink in the exact solution.
- The two-term approximation (u_b) is significantly closer to the exact solution, especially at $x = L$.

From Figure 3:

- The exact stress is piecewise constant with a jump from $2P/A$ to P/A at $x = L/2$.
- Part (a) gives a linearly increasing stress, completely missing the constant-stress nature of the solution.
- Part (b) gives a parabolic stress that peaks at $x = L/2$ with $\sigma_b(L/2) = 15P/(8A) = 1.875\,P/A$, which is between the two exact values ($2P/A$ and P/A). The two-term stress "averages" the jump rather than capturing it.
- Both approximations give $\sigma(0) = 0$, which is a direct consequence of the missing a_1x term.

8.5 Effect of Restoring the Linear Term

If the trial function were $u(x) = a_1x + a_2x^2 + a_3x^3$ (a complete cubic), the basis would be able to capture the constant-strain mode, and the Rayleigh-Ritz approximation would satisfy $\varepsilon(0) \neq 0$. The solution would be significantly improved, with much better stress predictions near the fixed end. This underscores Cook's observation that *completeness demands that the lowest-order admissible terms be included.*

9 Problem 2: Bilinear Rectangular Finite Element

Derive $u_i(s,t)$ for a $2a \times 2b$ rectangular element. Einstein summation implied; $i,j,k,l \in \{1,2\}$; $I,J \in \{1,2,3,4\}$; $r \in \{1,2,3,4\}$; $(\cdot)_{,j} = \partial(\cdot)/\partial x_j$, $x_1 \equiv s$, $x_2 \equiv t$.

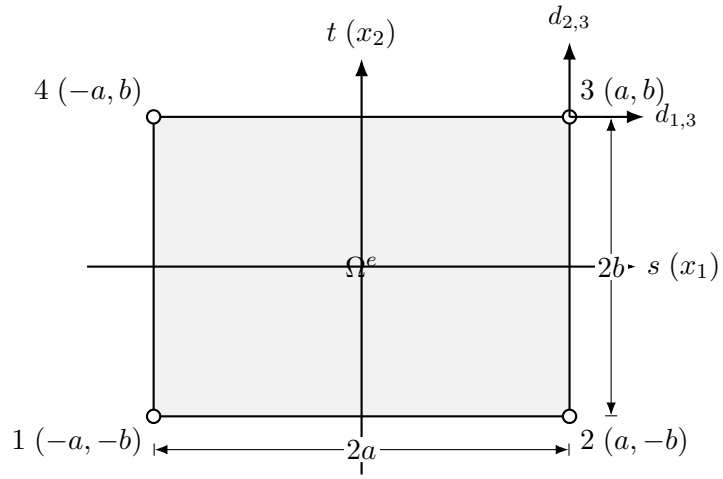

Figure 4: Rectangular element $\Omega^e = [-a,a] \times [-b,b]$; nodes numbered counter-clockwise. Each node carries two displacement DOFs d_{iI}.

9.1 Potential Energy and Rayleigh-Ritz Equations

Let $\varepsilon_{ij} = \frac{1}{2}(u_{i,j} + u_{j,i})$, $\sigma_{ij} = C_{ijkl}\,\varepsilon_{kl}$, with $C_{ijkl} = C_{jikl} = C_{ijlk} = C_{klij}$.

$$\Pi_p[u_i] = \frac{1}{2}\int_\Omega C_{ijkl}\,u_{i,j}\,u_{k,l}\,d\Omega - \int_\Omega f_i\,u_i\,d\Omega - \int_{\Gamma_N} \bar{t}_i\,u_i\,d\Gamma \tag{27}$$

Let $u_i^h = N_I\,d_{iI}$, $\varepsilon_{ij}^h = \frac{1}{2}(N_{I,j}\,d_{iI} + N_{I,i}\,d_{jI})$. Stationarity $\partial\Pi_p/\partial d_{iI} = 0$ gives

$$K_{iI,kJ}\,d_{kJ} = F_{iI} \tag{28}$$

$$K_{iI,kJ} = \int_{\Omega^e} C_{ijkl}\,N_{I,j}\,N_{J,l}\,d\Omega, \qquad F_{iI} = \int_{\Omega^e} f_i\,N_I\,d\Omega + \int_{\Gamma_N^e} \bar{t}_i\,N_I\,d\Gamma \tag{29}$$

9.2 Derivation of the Shape Functions

Polynomial basis. $p_r(s,t)$: $p_1 = 1$, $p_2 = s$, $p_3 = t$, $p_4 = st$ (4 monomials, 4 nodes).

Field approximation. Let $u_i(s,t) = p_r\,\alpha_{ir}$. At node I:

$$d_{iI} = p_r(s_I,t_I)\,\alpha_{ir} = A_{Ir}\,\alpha_{ir}$$

where $A_{Ir} := p_r(s_I,t_I)$:

$$[A_{Ir}] = \begin{bmatrix} p_1(s_1,t_1) & p_2(s_1,t_1) & p_3(s_1,t_1) & p_4(s_1,t_1) \\ p_1(s_2,t_2) & p_2(s_2,t_2) & p_3(s_2,t_2) & p_4(s_2,t_2) \\ p_1(s_3,t_3) & p_2(s_3,t_3) & p_3(s_3,t_3) & p_4(s_3,t_3) \\ p_1(s_4,t_4) & p_2(s_4,t_4) & p_3(s_4,t_4) & p_4(s_4,t_4) \end{bmatrix} = \begin{bmatrix} 1 & -a & -b & ab \\ 1 & a & -b & -ab \\ 1 & a & b & ab \\ 1 & -a & b & -ab \end{bmatrix}. \tag{30}$$

Inversion. $\alpha_{ir} = (A^{-1})_{rI}\, d_{iI}$. Row-reduce $[A \mid \mathbf{I}_4]$:

$$R_2 \leftarrow R_2 - R_1, \quad R_3 \leftarrow R_3 - R_1, \quad R_4 \leftarrow R_4 - R_1$$

$$\longrightarrow \begin{bmatrix} 1 & -a & -b & ab \\ 0 & 2a & 0 & -2ab \\ 0 & 2a & 2b & 0 \\ 0 & 0 & 2b & -2ab \end{bmatrix}$$

$$R_3 \leftarrow R_3 - R_2, \qquad R_4 \leftarrow R_4 - R_3$$

$$\longrightarrow \begin{bmatrix} 1 & -a & -b & ab \\ 0 & 2a & 0 & -2ab \\ 0 & 0 & 2b & 2ab \\ 0 & 0 & 0 & -4ab \end{bmatrix}$$

Hence $\det[A] = 1 \cdot 2a \cdot 2b \cdot (-4ab) = -16a^2b^2 \neq 0$. Back-substituting:

$$[(A^{-1})_{rI}] = \frac{1}{4} \begin{bmatrix} 1 & 1 & 1 & 1 \\ -\frac{1}{a} & \frac{1}{a} & \frac{1}{a} & -\frac{1}{a} \\ -\frac{1}{b} & -\frac{1}{b} & \frac{1}{b} & \frac{1}{b} \\ \frac{1}{ab} & -\frac{1}{ab} & \frac{1}{ab} & -\frac{1}{ab} \end{bmatrix} \tag{31}$$

Check: $A_{1r}(A^{-1})_{r1} = \frac{1}{4}+\frac{1}{4}+\frac{1}{4}+\frac{1}{4} = 1$, $A_{1r}(A^{-1})_{r2} = \frac{1}{4}-\frac{1}{4}+\frac{1}{4}-\frac{1}{4} = 0$. $\Rightarrow$ $A_{Ir}(A^{-1})_{rJ} = \delta_{IJ}$. ✓

Shape functions. $u_i = p_r\,(A^{-1})_{rI}\, d_{iI} = N_I\, d_{iI}$, where $N_I = p_r(A^{-1})_{rI}$. Column by column:

$I = 1$:

$$N_1 = [1,\ s,\ t,\ st]\ \frac{1}{4}\begin{Bmatrix} 1 \\ -1/a \\ -1/b \\ 1/(ab) \end{Bmatrix} = \frac{1}{4}\left(1 - \frac{s}{a} - \frac{t}{b} + \frac{st}{ab}\right)$$

$$= \frac{1}{4ab}\big(ab - sb - at + st\big) = \frac{(a-s)(b-t)}{4ab}$$

$I = 2$:

$$N_2 = [1,\ s,\ t,\ st]\ \frac{1}{4}\begin{Bmatrix} 1 \\ 1/a \\ -1/b \\ -1/(ab) \end{Bmatrix} = \frac{1}{4}\left(1 + \frac{s}{a} - \frac{t}{b} - \frac{st}{ab}\right)$$

$$= \frac{1}{4ab}\big(ab + sb - at - st\big) = \frac{(a+s)(b-t)}{4ab}$$

$I = 3$:

$$N_3 = [1,\ s,\ t,\ st]\ \frac{1}{4}\begin{Bmatrix} 1 \\ 1/a \\ 1/b \\ 1/(ab) \end{Bmatrix} = \frac{1}{4}\left(1 + \frac{s}{a} + \frac{t}{b} + \frac{st}{ab}\right)$$

$$= \frac{1}{4ab}\big(ab + sb + at + st\big) = \frac{(a+s)(b+t)}{4ab}$$

$I = 4$:

$$N_4 = [1,\ s,\ t,\ st]\ \frac{1}{4}\begin{Bmatrix} 1 \\ -1/a \\ 1/b \\ -1/(ab) \end{Bmatrix} = \frac{1}{4}\left(1 - \frac{s}{a} + \frac{t}{b} - \frac{st}{ab}\right)$$

$$= \frac{1}{4ab}\big(ab - sb + at - st\big) = \frac{(a-s)(b+t)}{4ab}$$

Compact form. Let $\xi_I := s_I/a \in \{-1,+1,+1,-1\}$, $\eta_I := t_I/b \in \{-1,-1,+1,+1\}$:

$$\boxed{N_I(s,t) = \frac{(a+\xi_I\, s)(b+\eta_I\, t)}{4ab} = \frac{1}{4}\left(1+\xi_I\frac{s}{a}\right)\left(1+\eta_I\frac{t}{b}\right)} \tag{32}$$

Explicitly:

$$N_1 = \frac{(a-s)(b-t)}{4ab}, \quad N_2 = \frac{(a+s)(b-t)}{4ab}, \quad N_3 = \frac{(a+s)(b+t)}{4ab}, \quad N_4 = \frac{(a-s)(b+t)}{4ab}. \tag{33}$$

Kronecker-delta. $s_J = \xi_J a$, $t_J = \eta_J b$:

$$N_I(s_J,t_J) = \frac{(1+\xi_I\xi_J)(1+\eta_I\eta_J)}{4} = \begin{cases} 1 & I = J \ \ (\xi_I\xi_J = \eta_I\eta_J = 1) \\ 0 & I \neq J \ \ \text{(at least one factor vanishes)} \end{cases} = \delta_{IJ}.\ \checkmark$$

Partition of unity.

$$\begin{aligned}
\sum_{I=1}^{4} N_I &= \frac{1}{4}\sum_{I=1}^{4}\left(1+\xi_I\frac{s}{a}\right)\left(1+\eta_I\frac{t}{b}\right) \\
&= \frac{1}{4}\sum_{I=1}^{4}\left[1+\xi_I\frac{s}{a}+\eta_I\frac{t}{b}+\xi_I\eta_I\frac{st}{ab}\right] \\
&= \frac{1}{4}\left[\underbrace{\sum_I 1}_{=4} + \frac{s}{a}\underbrace{\sum_I \xi_I}_{-1+1+1-1=0} + \frac{t}{b}\underbrace{\sum_I \eta_I}_{-1-1+1+1=0} + \frac{st}{ab}\underbrace{\sum_I \xi_I\eta_I}_{1-1+1-1=0}\right] \\
&= \frac{1}{4}\cdot 4 = 1
\end{aligned}$$

$$\sum_{I=1}^{4} N_I(s,t) = 1 \qquad \forall\ (s,t) \in \Omega^e.\ \checkmark \tag{34}$$

9.3 Displacement Interpolation

$$\boxed{u_i(s,t) = N_I(s,t)\ d_{iI} = \frac{(a+\xi_I\, s)(b+\eta_I\, t)}{4ab}\ d_{iI}, \qquad i \in \{1,2\},\ I \text{ summed 1–4}} \tag{35}$$

Expanded ($u \equiv u_1$, $v \equiv u_2$):

$$\begin{aligned} u(s,t) &= N_1\, u_1 + N_2\, u_2 + N_3\, u_3 + N_4\, u_4 \\ v(s,t) &= N_1\, v_1 + N_2\, v_2 + N_3\, v_3 + N_4\, v_4 \end{aligned} \tag{36}$$

Matrix form ($\{\mathbf{d}\} = \{u_1, v_1, u_2, v_2, u_3, v_3, u_4, v_4\}^T$):

$$\begin{Bmatrix} u \\ v \end{Bmatrix} = \underbrace{\begin{bmatrix} N_1 & 0 & N_2 & 0 & N_3 & 0 & N_4 & 0 \\ 0 & N_1 & 0 & N_2 & 0 & N_3 & 0 & N_4 \end{bmatrix}}_{[\mathbf{N}]\ (2\times 8)} \{\mathbf{d}\} \tag{37}$$

9.4 Strain–Displacement Relation and Element Stiffness

Gradients:

$$N_{I,1} = \frac{\partial N_I}{\partial s} = \frac{\xi_I(b + \eta_I t)}{4ab}, \qquad N_{I,2} = \frac{\partial N_I}{\partial t} = \frac{\eta_I(a + \xi_I s)}{4ab}. \tag{38}$$

In Voigt notation $\{\boldsymbol{\varepsilon}\} = \{\varepsilon_{11}, \varepsilon_{22}, 2\varepsilon_{12}\}^T = [\mathbf{B}]\{\mathbf{d}\}$, the 3×2 sub-block for node I:

$$[\mathbf{B}_I] = \begin{bmatrix} N_{I,1} & 0 \\ 0 & N_{I,2} \\ N_{I,2} & N_{I,1} \end{bmatrix} = \frac{1}{4ab} \begin{bmatrix} \xi_I(b + \eta_I t) & 0 \\ 0 & \eta_I(a + \xi_I s) \\ \eta_I(a + \xi_I s) & \xi_I(b + \eta_I t) \end{bmatrix} \tag{39}$$

$[\mathbf{B}] = [\mathbf{B}_1\ \mathbf{B}_2\ \mathbf{B}_3\ \mathbf{B}_4]$ (3×8, varies linearly in s, t). Element stiffness ($h = 1$):

$$[\mathbf{k}^e] = \int_{\Omega^e} [\mathbf{B}]^T [\mathbf{C}][\mathbf{B}]\, dA = \int_{-b}^{b}\int_{-a}^{a} [\mathbf{B}]^T [\mathbf{C}][\mathbf{B}]\, ds\, dt \tag{40}$$

In index notation:

$$k_{iI,kJ} = \int_{-b}^{b}\int_{-a}^{a} C_{ijkl}\, N_{I,j}(s,t)\, N_{J,l}(s,t)\, ds\, dt. \tag{41}$$

This is the 8×8 element stiffness for the $2a \times 2b$ bilinear rectangle.

9.5 Natural (Isoparametric) Coordinates

Coordinate mapping. Let $\hat{\Omega} = [-1, 1]^2$ (parent domain). Map to Ω^e:

$$s = a\,\zeta, \qquad t = b\,\eta \tag{42}$$

so that

$$\zeta = \frac{s}{a} \in [-1, 1], \qquad \eta = \frac{t}{b} \in [-1, 1].$$

Nodes: $(\zeta_I, \eta_I) = (\xi_I, \eta_I) = \{(-1, -1), (+1, -1), (+1, +1), (-1, +1)\}$.

Jacobian.

$$[\mathbf{J}] = \begin{bmatrix} \dfrac{\partial s}{\partial \zeta} & \dfrac{\partial t}{\partial \zeta} \\ \dfrac{\partial s}{\partial \eta} & \dfrac{\partial t}{\partial \eta} \end{bmatrix} = \begin{bmatrix} a & 0 \\ 0 & b \end{bmatrix} \tag{43}$$

$$\det[\mathbf{J}] = a \cdot b - 0 \cdot 0 = ab \tag{44}$$

$$[\mathbf{J}]^{-1} = \frac{1}{ab} \begin{bmatrix} b & 0 \\ 0 & a \end{bmatrix} = \begin{bmatrix} 1/a & 0 \\ 0 & 1/b \end{bmatrix} \tag{45}$$

Shape functions in (ζ, η).

$$\boxed{\hat{N}_I(\zeta,\eta) = \frac{1}{4}(1+\xi_I\,\zeta)(1+\eta_I\,\eta)} \tag{46}$$

Explicitly:

$$\begin{aligned}
\hat{N}_1 &= \tfrac{1}{4}(1-\zeta)(1-\eta)\\
\hat{N}_2 &= \tfrac{1}{4}(1+\zeta)(1-\eta)\\
\hat{N}_3 &= \tfrac{1}{4}(1+\zeta)(1+\eta)\\
\hat{N}_4 &= \tfrac{1}{4}(1-\zeta)(1+\eta)
\end{aligned}$$

Equivalence. $\zeta = s/a$, $\eta = t/b$:

$$\begin{aligned}
\hat{N}_I\left(\frac{s}{a},\frac{t}{b}\right) &= \frac{1}{4}\left(1+\xi_I\frac{s}{a}\right)\left(1+\eta_I\frac{t}{b}\right)\\
&= \frac{1}{4}\cdot\frac{a+\xi_I s}{a}\cdot\frac{b+\eta_I t}{b} = \frac{(a+\xi_I s)(b+\eta_I t)}{4ab} = N_I(s,t)\ \checkmark
\end{aligned}$$

Gradient transformation.

$$\begin{Bmatrix} \dfrac{\partial N_I}{\partial s} \\ \dfrac{\partial N_I}{\partial t} \end{Bmatrix} = [\mathbf{J}]^{-1} \begin{Bmatrix} \dfrac{\partial \hat{N}_I}{\partial \zeta} \\ \dfrac{\partial \hat{N}_I}{\partial \eta} \end{Bmatrix} \tag{47}$$

Parent-domain derivatives: $\partial\hat{N}_I/\partial\zeta = \xi_I(1+\eta_I\eta)/4$, $\partial\hat{N}_I/\partial\eta = \eta_I(1+\xi_I\zeta)/4$. Applying $[\mathbf{J}]^{-1}$:

$$\begin{aligned}
\frac{\partial N_I}{\partial s} &= \frac{1}{a}\cdot\frac{\xi_I}{4}(1+\eta_I\eta) = \frac{\xi_I(1+\eta_I\eta)}{4a} = \frac{\xi_I(b+\eta_I t)}{4ab}\ \checkmark\\
\frac{\partial N_I}{\partial t} &= \frac{1}{b}\cdot\frac{\eta_I}{4}(1+\xi_I\zeta) = \frac{\eta_I(1+\xi_I\zeta)}{4b} = \frac{\eta_I(a+\xi_I s)}{4ab}\ \checkmark
\end{aligned}$$

These match Equation (38) exactly.

Stiffness in (ζ, η). $ds\,dt = ab\,d\zeta\,d\eta$:

$$[\mathbf{k}^e] = \int_{-1}^{1}\int_{-1}^{1} [\mathbf{B}(\zeta,\eta)]^T\,[\mathbf{C}]\,[\mathbf{B}(\zeta,\eta)]\ \det[\mathbf{J}]\,d\zeta\,d\eta \tag{48}$$

$\det[\mathbf{J}] = ab$ is constant:

$$[\mathbf{k}^e] = ab\int_{-1}^{1}\int_{-1}^{1} [\mathbf{B}(\zeta,\eta)]^T\,[\mathbf{C}]\,[\mathbf{B}(\zeta,\eta)]\,d\zeta\,d\eta \tag{49}$$

2×2 Gauss quadrature ($\zeta_g, \eta_g = \pm 1/\sqrt{3}$, $w_g = 1$) integrates exactly:

$$[\mathbf{k}^e] = ab\sum_{g=1}^{4} w_g\,[\mathbf{B}(\zeta_g,\eta_g)]^T\,[\mathbf{C}]\,[\mathbf{B}(\zeta_g,\eta_g)] \tag{50}$$

In practice: work in (ζ, η), never form $[A]^{-1}$, evaluate via Gauss quadrature.

9.6 Discussion of Problem 2

The shape functions in Equation (32) have a tensor-product structure: $N_I(s,t) = L^{(s)}_{m(I)}(s)\, L^{(t)}_{n(I)}(t)$, where $L^{(s)}_1 = (a-s)/(2a)$, $L^{(s)}_2 = (a+s)/(2a)$ are the 1-D linear shape functions in s, and similarly for t. This is a direct extension of the bar element interpolation (Cook, Section 3.8, Eq. 3.8-8) into two dimensions.

The bilinear basis $\{1, s, t, st\}$ is *complete to degree 1* in Pascal's triangle (all terms with $s^m t^n$, $m+n \leq 1$, are present), plus the cross-term st. Completeness to degree 1 guarantees constant-strain (and hence constant-stress) capability: any state $u_i = a_i + b_{ij}x_j$ with constant b_{ij} can be represented exactly. The partition-of-unity property ($\sum N_I = 1$) ensures that rigid-body translations are contained in the approximation space.

The strain–displacement matrix $[\mathbf{B}]$ is not constant over Ω^e; it varies linearly in s and t. This is a key difference from the 1-D bar, where $[\mathbf{B}]$ is constant. As a consequence, the integrand of the stiffness integral Equation (40) is quadratic in s and t, and the analytical evaluation requires integration of polynomial terms up to s^2t^2 over the rectangular domain — possible in closed form, but often delegated to 2×2 Gauss quadrature in practice.

The element has 8 DOFs and 3 independent strain components, so the system is well-posed. It possesses 3 rigid-body modes (u-translation, v-translation, and in-plane rotation for small angles) and 5 deformation modes, consistent with $\text{rank}([\mathbf{k}^e]) = 5$.

10 Numerical Validation

The Rayleigh-Ritz solutions from Question 1 and the element stiffness from Question 2 are validated against two independent FEM implementations: **OpenSees** (via `openseespy`) and **FemLab-Python** (in-house FEM library).

10.1 OpenSees Validation — Question 1

A 1-D bar with 20 linear truss elements ($\Delta x = L/20$) is modelled in OpenSees with the same loading (P at $x = L/2$ and $x = L$) and boundary condition ($u(0) = 0$). Since the exact displacement is piecewise linear, the linear FEM solution coincides with the exact solution to machine precision.

$\bar{x}$	Exact	OpenSees	RR (a)	RR (b)	FEM err (%)
0.00	0.0000	0.0000	0.0000	0.0000	0.000
0.25	0.5000	0.5000	0.0586	0.1953	0.000
0.50	1.0000	1.0000	0.2344	0.6250	0.000
0.75	1.2500	1.2500	0.5273	1.0547	0.000
1.00	1.5000	1.5000	0.9375	1.2500	0.000

Table 3: Normalised displacement $\bar{u} = u \cdot EA/(PL)$: OpenSees FEM vs. exact and Rayleigh-Ritz.

$\bar{x}$	Exact	OpenSees	RR (a)	RR (b)
0.00	2.0000	2.0000	0.0000	0.0000
0.25	2.0000	2.0000	0.4688	1.4062
0.50	2.0000	2.0000	0.9375	1.8750
0.75	1.0000	1.0000	1.4062	1.4062
1.00	1.0000	1.0000	1.8750	0.0000

Table 4: Normalised stress $\bar{\sigma} = \sigma \cdot A/P$: OpenSees FEM vs. exact and Rayleigh-Ritz.

The FEM solution matches the exact solution with $\max |u_{\text{FEM}} - u_{\text{exact}}| = \mathcal{O}(10^{-14})$, confirming both the exact solution derivation and the FEM implementation. The large discrepancies between the Rayleigh-Ritz approximations and the FEM/exact results — particularly for stress near $x = 0$ — are a direct consequence of the missing $a_1 x$ term discussed in Section 8.

10.2 FemLab Validation — Question 1

The same 1-D bar problem is solved using the FemLab-Python library (`kbar` element) with 20 elements.

$\bar{x}$	Exact	FemLab	RR (a)	RR (b)	FEM err (%)
0.00	0.0000	0.0000	0.0000	0.0000	0.000
0.25	0.5000	0.5000	0.0586	0.1953	0.000
0.50	1.0000	1.0000	0.2344	0.6250	0.000
0.75	1.2500	1.2500	0.5273	1.0547	0.000
1.00	1.5000	1.5000	0.9375	1.2500	0.000

Table 5: Normalised displacement: FemLab vs. exact and Rayleigh-Ritz.

FemLab results are identical to OpenSees and to the exact solution, providing independent cross-validation.

10.3 FemLab Validation — Question 2

The element stiffness matrix $[\mathbf{k}^e]$ for a single 2×2 plane-stress Q4 element ($E = 100$, $\nu = 0.3$, $h = 1$) is computed using FemLab's `keq4e` routine and compared against the analytical 2×2 Gauss quadrature result from Equation (49):

Check	**Result**
$\|[\mathbf{k}^e]_{\text{FemLab}} - [\mathbf{k}^e]_{\text{analytical}}\|$	2.76×10^{-14}
Relative error	1.43×10^{-16}
Zero eigenvalues (rigid-body modes)	3
Positive eigenvalues (deformation modes)	5
rank($[\mathbf{k}^e]$)	5
$\|[\mathbf{k}^e]\,\mathbf{d}_{\text{rigid}}\|$	1.33×10^{-14}

Table 6: FemLab element stiffness verification for the $2a \times 2b$ bilinear rectangle.

The FemLab Q4 element stiffness matches the analytical Gauss-quadrature result to machine precision. The eigenvalue analysis confirms the expected rank of 5 (three rigid-body modes in 2-D: two translations and one rotation) and five independent deformation modes.

11 Convergence Study: n-Term Polynomial

11.1 Function Space and Admissibility

Let $\mathcal{V}$ be the space of kinematically admissible displacement fields:

$$\mathcal{V} = \left\{ v \in H^1(0, L) \ : \ v(0) = 0 \right\} \tag{51}$$

where $H^1(0, L)$ is the Sobolev space of functions with square-integrable first derivatives, equipped with the *energy inner product*

$$\langle u, v \rangle_E = EA \int_0^L u_{,x}\, v_{,x}\, dx \tag{52}$$

and the induced *energy norm* $\|u\|_E = \sqrt{\langle u, u \rangle_E}$.

11.2 General n-Term Trial Function

Let the n-dimensional trial subspace $\mathcal{V}_n^h \subset \mathcal{V}$ be spanned by the monomial basis

$$\phi_i(x) = x^{i+1}, \qquad i = 1, 2, \ldots, n \tag{53}$$

so that the index i labels the *basis function number* while the polynomial degree is $i + 1$. Every ϕ_i satisfies the essential BC $\phi_i(0) = 0$. Note: $\phi_1 = x^2$, corresponding to parts (a)–(b); the linear term x^1 remains **omitted**.

The trial displacement is

$$u^h(x) = \sum_{i=1}^{n} a_i\, \phi_i(x) = a_1\, x^2 + a_2\, x^3 + \cdots + a_n\, x^{n+1}, \qquad a_i \in \mathbb{R} \tag{54}$$

and the corresponding strain:

$$\varepsilon^h(x) = u^h_{,x} = \sum_{i=1}^{n} a_i\, \phi_i'(x) = 2\, a_1\, x + 3\, a_2\, x^2 + \cdots + (n{+}1)\, a_n\, x^n \tag{55}$$

11.3 Total Potential Energy

Substituting Equation (54) into the functional Equation (1):

Strain energy.

$$
\begin{aligned}
U &= \frac{EA}{2}\int_0^L \left(u^h_{,x}\right)^2 dx = \frac{EA}{2}\int_0^L \left(\sum_{i=1}^{n} a_i\,(i{+}1)\,x^i\right)\left(\sum_{j=1}^{n} a_j\,(j{+}1)\,x^j\right) dx \\
&= \frac{EA}{2}\sum_{i=1}^{n}\sum_{j=1}^{n} a_i\,a_j\,(i{+}1)(j{+}1)\int_0^L x^{i+j}\,dx \\
&= \frac{EA}{2}\sum_{i=1}^{n}\sum_{j=1}^{n} a_i\,a_j\,(i{+}1)(j{+}1)\cdot\frac{L^{i+j+1}}{i+j+1} \\
&= \frac{1}{2}\begin{Bmatrix} a_1 & a_2 & \cdots & a_n \end{Bmatrix}\begin{bmatrix} K_{11} & K_{12} & \cdots & K_{1n} \\ K_{21} & K_{22} & \cdots & K_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ K_{n1} & K_{n2} & \cdots & K_{nn} \end{bmatrix}\begin{Bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{Bmatrix}
\end{aligned}
\tag{56}
$$

Identification of the stiffness matrix [K]. The (i,j) entry of the $n\times n$ stiffness matrix is read off from Equation (56):

$$
\boxed{K_{ij} = EA\,\frac{(i+1)(j+1)}{i+j+1}\,L^{i+j+1}, \qquad i,j = 1,2,\ldots,n}
\tag{57}
$$

In full matrix form:

$$
[\mathbf{K}] = EA\begin{bmatrix}
\frac{2\cdot 2}{3}L^3 & \frac{2\cdot 3}{4}L^4 & \frac{2\cdot 4}{5}L^5 & \cdots & \frac{2(n{+}1)}{n{+}2}L^{n+2} \\
\frac{3\cdot 2}{4}L^4 & \frac{3\cdot 3}{5}L^5 & \frac{3\cdot 4}{6}L^6 & \cdots & \frac{3(n{+}1)}{n{+}3}L^{n+3} \\
\frac{4\cdot 2}{5}L^5 & \frac{4\cdot 3}{6}L^6 & \frac{4\cdot 4}{7}L^7 & \cdots & \frac{4(n{+}1)}{n{+}4}L^{n+4} \\
\vdots & \vdots & \vdots & \ddots & \vdots \\
\frac{(n{+}1)\cdot 2}{n{+}2}L^{n+2} & \frac{(n{+}1)\cdot 3}{n{+}3}L^{n+3} & \frac{(n{+}1)\cdot 4}{n{+}4}L^{n+4} & \cdots & \frac{(n{+}1)^2}{2n{+}1}L^{2n+1}
\end{bmatrix}
\tag{58}
$$

Symmetry check. Exchanging $i \leftrightarrow j$: $K_{ji} = EA\,\frac{(j+1)(i+1)}{j+i+1}\,L^{j+i+1} = K_{ij}$ ✓ $\implies$ $[\mathbf{K}]^T = [\mathbf{K}]$.

Potential of external forces.

$$
\begin{aligned}
\Omega &= -P\,u^h\!\left(\frac{L}{2}\right) - P\,u^h(L) \\
&= -P\sum_{i=1}^{n} a_i\left[\left(\frac{L}{2}\right)^{i+1} + L^{i+1}\right] \\
&= -\begin{Bmatrix} F_1 & F_2 & \cdots & F_n \end{Bmatrix}\begin{Bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{Bmatrix}
\end{aligned}
\tag{59}
$$

Identification of the force vector $\{\mathbf{F}\}$. The i-th component of the $n \times 1$ force vector:

$$\boxed{F_i = P\left[\left(\frac{L}{2}\right)^{i+1} + L^{i+1}\right] = P\,L^{i+1}\left(\frac{1}{2^{i+1}} + 1\right), \qquad i = 1, 2, \ldots, n} \tag{60}$$

In column-vector form:

$$\{\mathbf{F}\} = P\begin{Bmatrix} \left(\frac{L}{2}\right)^2 + L^2 \\ \left(\frac{L}{2}\right)^3 + L^3 \\ \left(\frac{L}{2}\right)^4 + L^4 \\ \vdots \\ \left(\frac{L}{2}\right)^{n+1} + L^{n+1} \end{Bmatrix} = P\begin{Bmatrix} \frac{5L^2}{4} \\ \frac{9L^3}{8} \\ \frac{17L^4}{16} \\ \vdots \\ \left(\frac{1}{2^{n+1}} + 1\right) L^{n+1} \end{Bmatrix} \tag{61}$$

Total potential energy.

$$\Pi_p(a_1, \ldots, a_n) = \frac{1}{2}\begin{Bmatrix} a_1 & \cdots & a_n \end{Bmatrix}\begin{bmatrix} K_{11} & \cdots & K_{1n} \\ \vdots & \ddots & \vdots \\ K_{n1} & \cdots & K_{nn} \end{bmatrix}\begin{Bmatrix} a_1 \\ \vdots \\ a_n \end{Bmatrix} - \begin{Bmatrix} F_1 & \cdots & F_n \end{Bmatrix}\begin{Bmatrix} a_1 \\ \vdots \\ a_n \end{Bmatrix} \tag{62}$$

11.4 Stationarity and the Rayleigh-Ritz System

Stationarity of Π_p with respect to each coefficient a_i $(i = 1, 2, \ldots, n)$.

Stationarity equations. Taking the energy derivative with respect to each a_i $(i = 1, 2, \ldots, n)$ yields:

$$\frac{\partial \Pi_p}{\partial a_i} = \sum_{j=1}^{n} K_{ij}\, a_j - F_i = 0 \tag{63}$$

Writing all n equations simultaneously as a full matrix system:

$$\boxed{\begin{bmatrix} K_{11} & K_{12} & K_{13} & \cdots & K_{1n} \\ K_{21} & K_{22} & K_{23} & \cdots & K_{2n} \\ K_{31} & K_{32} & K_{33} & \cdots & K_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ K_{n1} & K_{n2} & K_{n3} & \cdots & K_{nn} \end{bmatrix}\begin{Bmatrix} a_1 \\ a_2 \\ a_3 \\ \vdots \\ a_n \end{Bmatrix} = \begin{Bmatrix} F_1 \\ F_2 \\ F_3 \\ \vdots \\ F_n \end{Bmatrix}} \tag{64}$$

Row-by-row interpretation. The i-th equation reads:

$$K_{i1}\, a_1 + K_{i2}\, a_2 + K_{i3}\, a_3 + \cdots + K_{in}\, a_n = F_i$$

which is exactly $\partial \Pi_p / \partial a_i = 0$.

11.5 Explicit Matrix Entries

Normalized form. Let $\bar{x} = x/L$, $\bar{u} = u \cdot EA/(PL)$. Set $P = EA = L = 1$. With the row/column index shifted as $r = i + 1$, $c = j + 1$ $(r, c \in \{2, 3, \ldots, n{+}1\})$, Equations (57) and (60) become

$$K_{rc} = \frac{r\,c}{r + c - 1}, \qquad F_r = \frac{1}{2^r} + 1, \qquad r, c \in \{2, \ldots, n{+}1\} \tag{65}$$

Explicit construction for small n. Colour code: Part (a) ($n = 1$), Part (b) ($n = 2$), new terms.

$n = 1$: $[K_{11}]\{a_1\} = \{F_1\}$. Using Equations (57) and (60) with $i = j = 1$:

$$K_{11} = EA\,\frac{(1{+}1)(1{+}1)}{1{+}1{+}1}\,L^3 = \frac{4\,EAL^3}{3}, \qquad F_1 = P\left[\left(\frac{L}{2}\right)^2 + L^2\right] = \frac{5PL^2}{4}$$

$$a_1 = \frac{F_1}{K_{11}} = \frac{5PL^2}{4}\cdot\frac{3}{4\,EAL^3} = \frac{15\,P}{16\,EAL} \;=\; a_2 \text{ in Equation (8)} \;\checkmark$$

$n = 2$: 2×2 system (normalised, $P{=}EA{=}L{=}1$):

$$\begin{bmatrix} \frac{4}{3} & \frac{3}{2} \\ \frac{3}{2} & \frac{9}{5} \end{bmatrix} \begin{Bmatrix} a_1 \\ a_2 \end{Bmatrix} = \begin{Bmatrix} \frac{5}{4} \\ \frac{9}{8} \end{Bmatrix} \quad\Longrightarrow\quad a_1 = \frac{15}{4},\; a_2 = -\frac{5}{2} \;=\; \text{Equation (15)} \;\checkmark$$

$n = 3$:

$$\begin{bmatrix} \frac{4}{3} & \frac{6}{4} & \frac{8}{5} \\ \frac{6}{4} & \frac{9}{5} & \frac{12}{6} \\ \frac{8}{5} & \frac{12}{6} & \frac{16}{7} \end{bmatrix} \begin{Bmatrix} a_1 \\ a_2 \\ a_3 \end{Bmatrix} = \begin{Bmatrix} \frac{5}{4} \\ \frac{9}{8} \\ \frac{17}{16} \end{Bmatrix}$$

$n = 4$:

$$\begin{bmatrix} \frac{4}{3} & \frac{6}{4} & \frac{8}{5} & \frac{10}{6} \\ \frac{6}{4} & \frac{9}{5} & \frac{12}{6} & \frac{15}{7} \\ \frac{8}{5} & \frac{12}{6} & \frac{16}{7} & \frac{20}{8} \\ \frac{10}{6} & \frac{15}{7} & \frac{20}{8} & \frac{25}{9} \end{bmatrix} \begin{Bmatrix} a_1 \\ a_2 \\ a_3 \\ a_4 \end{Bmatrix} = \begin{Bmatrix} \frac{5}{4} \\ \frac{9}{8} \\ \frac{17}{16} \\ \frac{33}{32} \end{Bmatrix}$$

Submatrix nesting. Each $n \times n$ system is the **top-left submatrix** of all larger systems:

$$\underbrace{[K_{11}]}_{n=1} \subset \underbrace{\begin{bmatrix} K_{11} & K_{12} \\ K_{21} & K_{22} \end{bmatrix}}_{n=2\text{ (Part b)}} \subset \underbrace{\begin{bmatrix} K_{11} & K_{12} & K_{13} \\ K_{21} & K_{22} & K_{23} \\ K_{31} & K_{32} & K_{33} \end{bmatrix}}_{n=3} \subset \cdots$$

Part (a) is the $(1, 1)$ entry of $[\mathbf{K}]$; Part (b) is the 2×2 top-left block. Adding terms *never modifies* previous entries—only appends new rows/columns.

11.6 Displacement and Stress from the n-Term Solution

Given the coefficient column vector $\{\mathbf{a}\} = \{a_1, a_2, \ldots, a_n\}^T$ obtained from solving the matrix system derived above:

The displacement $u^h(x)$ can be written compactly as the dot product of a basis row vector and the coefficient vector:

$$u^h(x) = \begin{Bmatrix} x^2 & x^3 & \cdots & x^{n+1} \end{Bmatrix} \begin{Bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{Bmatrix} \tag{66}$$

Similarly, the stress $\sigma^h(x) = E\, u^h_{,x}$:

$$\sigma^h(x) = E \begin{Bmatrix} 2x & 3x^2 & \cdots & (n{+}1)x^n \end{Bmatrix} \begin{Bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{Bmatrix} \tag{67}$$

Recovering the formulations for small n. For $n = 1$ (equivalent to Part a), reading from the vectors with $a_1 = \frac{15P}{16EAL}$:

$$u^h_1(x) = \{x^2\} \left\{ \frac{15P}{16EAL} \right\} = \frac{15P}{16EAL}\, x^2$$

$$\sigma^h_1(x) = E\,\{2x\} \left\{ \frac{15P}{16EAL} \right\} = \frac{15Px}{8AL}$$

These exactly match Equation (9) and Equation (10).

For $n = 2$ (equivalent to Part b), substituting the coefficient vector from the 2×2 solution:

$$u^h_2(x) = \begin{Bmatrix} x^2 & x^3 \end{Bmatrix} \begin{Bmatrix} \dfrac{15P}{4EAL} \\ -\dfrac{5P}{2EAL^2} \end{Bmatrix} = \frac{15P}{4EAL}\, x^2 - \frac{5P}{2EAL^2}\, x^3$$

$$\sigma^h_2(x) = E \begin{Bmatrix} 2x & 3x^2 \end{Bmatrix} \begin{Bmatrix} \dfrac{15P}{4EAL} \\ -\dfrac{5P}{2EAL^2} \end{Bmatrix} = \frac{15P}{2AL}\, x - \frac{15P}{2AL^2}\, x^2$$

These exactly match Equation (16) and Equation (17).
Note: $\sigma^h(0) = 0$ for *every* n, since every term in the strain row vector contains x^i with $i \geq 1$.

11.7 Error Measures

Let $e_u = u - u^h$, $e_\sigma = \sigma - \sigma^h$. Define:

$$\|e_u\|_{L^2} = \left(\int_0^L (u - u^h)^2\, dx \right)^{1/2} \tag{68}$$

$$\|e_u\|_{L^\infty} = \max_{x \in [0,L]} |u(x) - u^h(x)| \tag{69}$$

$$\|e_\sigma\|_{L^2} = \left(\int_0^L (\sigma - \sigma^h)^2\, dx \right)^{1/2} \tag{70}$$

$$\|e_\sigma\|_{L^\infty} = \max_{x \in [0,L]} |\sigma(x) - \sigma^h(x)| \tag{71}$$

$$e_{\text{tip}} = \frac{|u^h(L) - u_{\text{exact}}(L)|}{u_{\text{exact}}(L)} \times 100\% \tag{72}$$

The energy norm relates to the stress L^2 norm: $\|e_u\|_E = \sqrt{EA}\, \|e_\sigma\|_{L^2} / \sqrt{E} = \sqrt{A/E}\, \|e_\sigma\|_{L^2}$.

11.8 Numerical Implementation

A Julia script assembles the $n \times n$ system **??** for $n = 1, 2, 3, 4, 5, 8, 12, 20, 30, 50$, solves via Cholesky ([**K**] is SPD), evaluates Equations (66) and (67) on a 500-point grid, computes the error norms Equation (68)–Equation (72) via trapezoidal quadrature ($N_q = 2000$), and exports tab-separated `.dat` files for pgfplots.

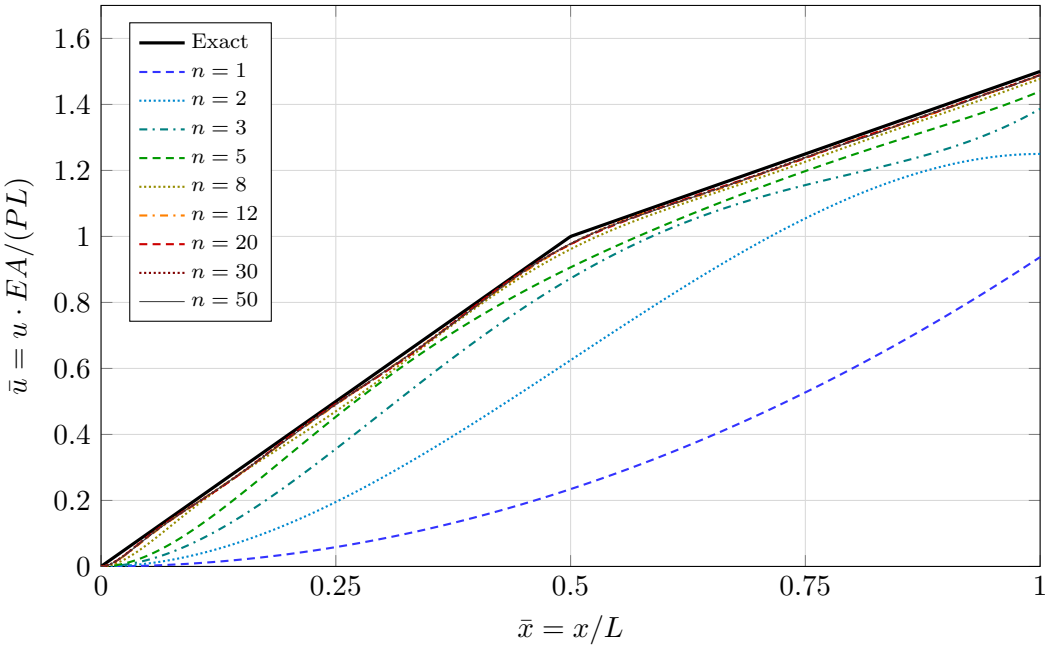

Figure 5: Displacement convergence: as n increases the polynomial approaches the exact piecewise-linear field.

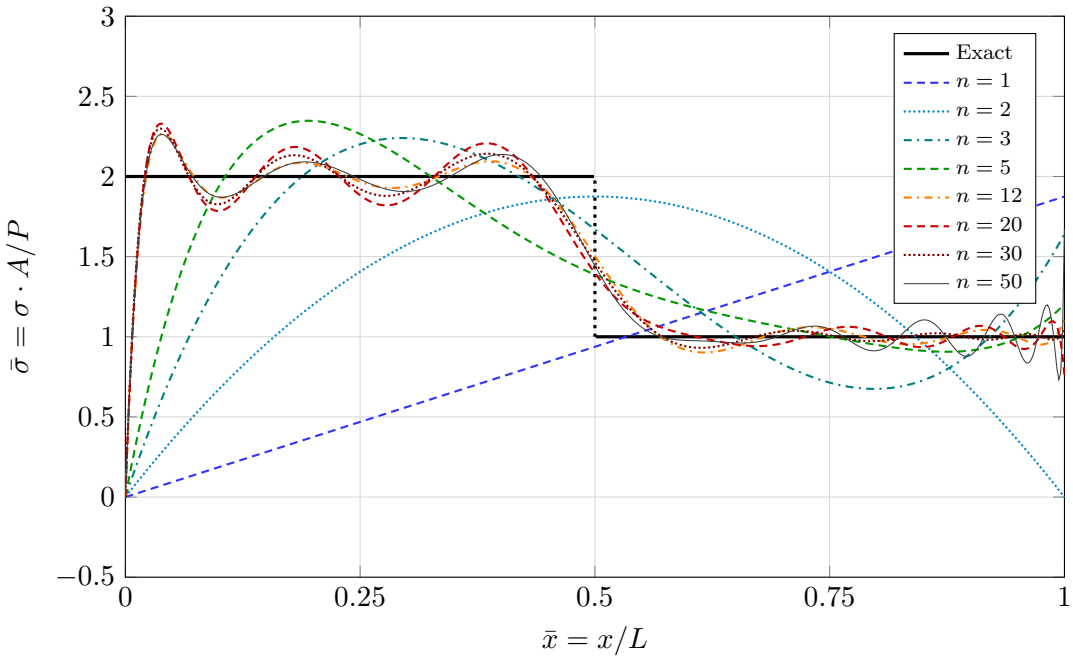

Figure 6: Stress convergence (Gibbs-like phenomenon): $\sigma(0) = 0$ persists for all n due to missing $a_1 x$; oscillations near $x = L/2$ do not decay.

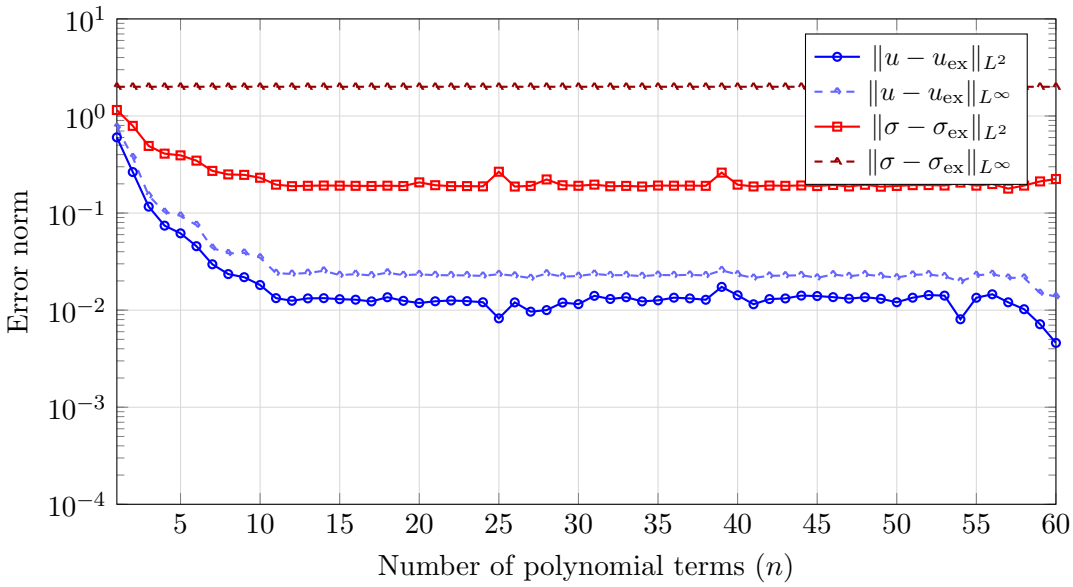

Figure 7: Error norms vs. n. Displacement errors decrease; $\|\sigma\|_{L^\infty} \approx 2$ persists ($\sigma(0)$ cannot be recovered).

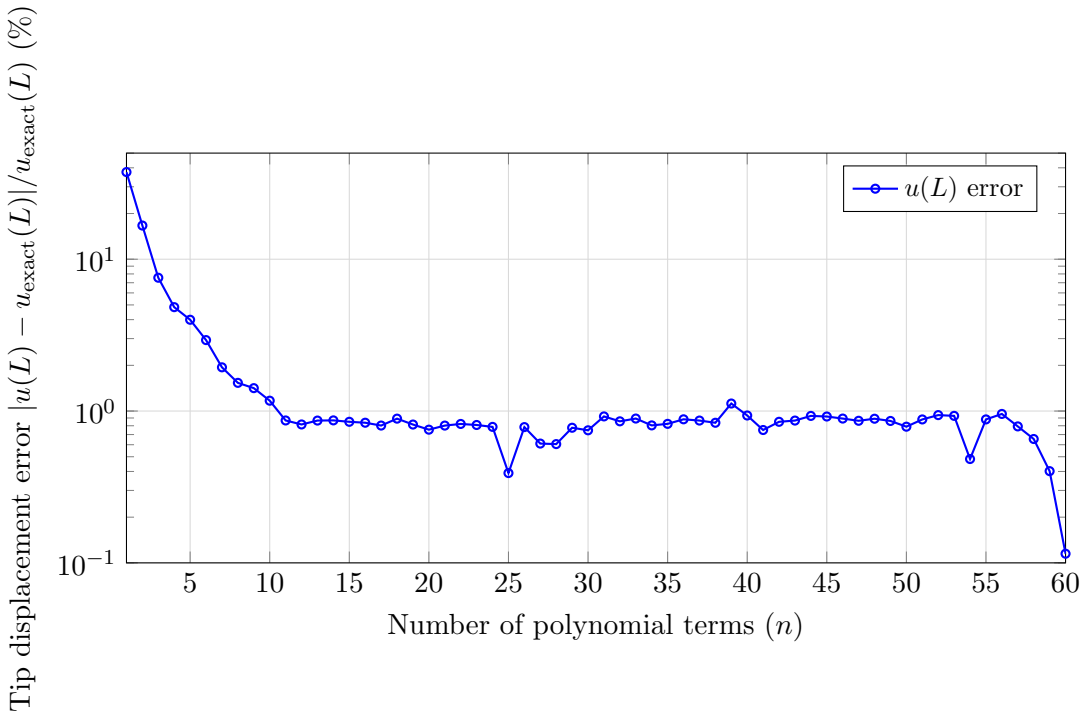

Figure 8: Relative error in tip displacement $u(L)$ vs. number of polynomial terms.

The displacement L^2 error decreases monotonically, confirming the Rayleigh-Ritz convergence theorem. However, the stress L^∞ norm plateaus at $\approx 2P/A$ — the exact stress at $x = 0$ that no polynomial $u = \sum_{k\geq 2} a_k x^k$ can capture (since $u_{,x}(0) = 0$ always). This is analogous to the Gibbs phenomenon in Fourier series: increasing the number of terms improves the fit everywhere *except* near the incompatible boundary, where oscillations persist.

11.9 Large-Scale Solve: n up to 10 000

The $n \times n$ stiffness matrix $K_{ij} = ij/(i+j-1)$ is a weighted Hilbert matrix. Its spectral condition number $\kappa_2(\mathbf{K})$ grows exponentially with n, eventually exceeding $1/\varepsilon_{\text{mach}} \approx 10^{16}$. A Julia script assembles and solves the system for n up to 10 000 using multi-threaded BLAS (16 CPU threads, Cholesky factorisation for SPD $\mathbf{K}$).

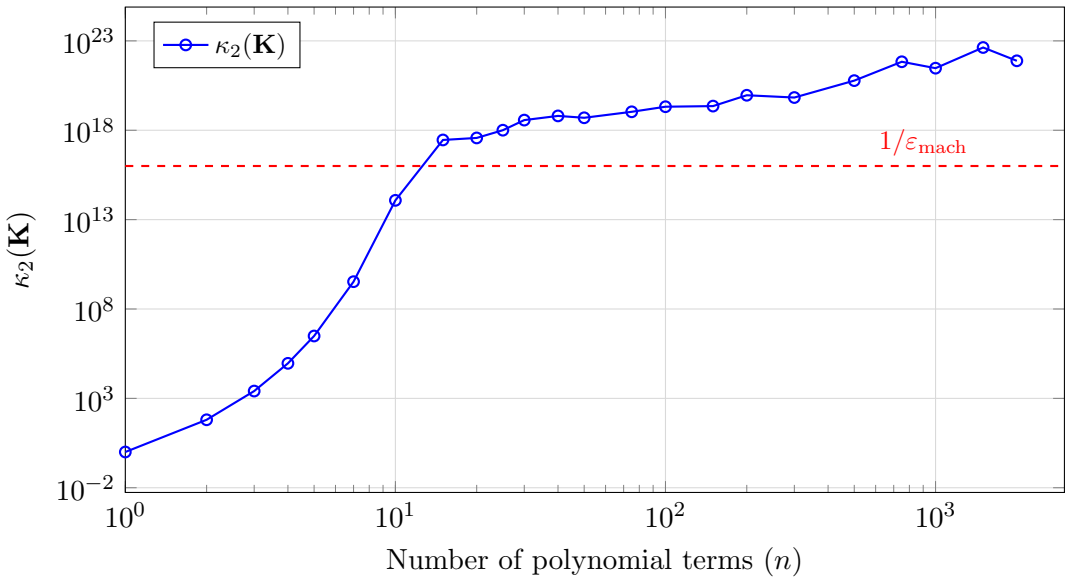

Figure 9: Spectral condition number of the stiffness matrix. The Hilbert-like structure drives κ_2 past 10^{16} (machine epsilon) by $n \approx 15$; beyond this, solutions lose all significant digits yet the Cholesky solver still returns finite answers.

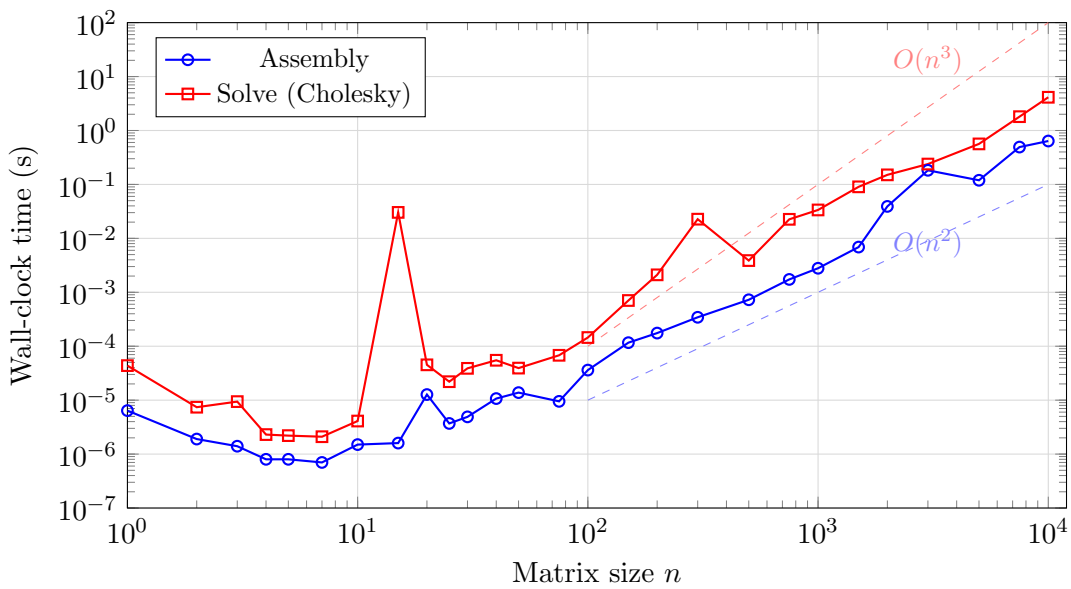

Figure 10: Assembly and solve time vs. n (Vectorized Assembly, CUDA GPU Solve for $n \geq 256$). Assembly scales as $O(n^2)$ (fill n^2 entries); the solve scales as $O(n^3)$ but highly offloads computation to the GPU.

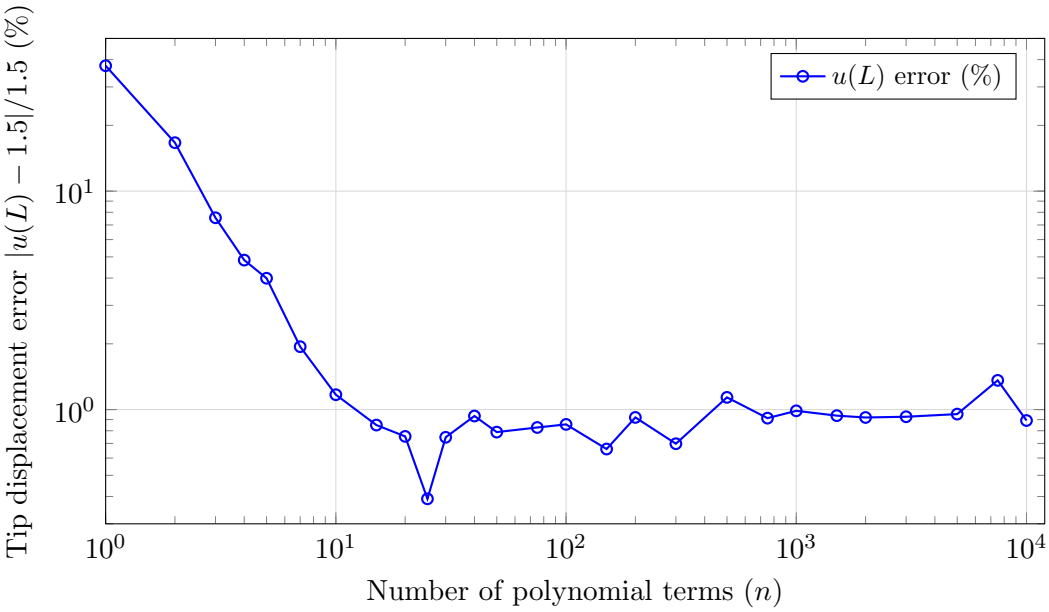

Figure 11: Tip-displacement error vs. n (log–log, up to $n = 10{,}000$). Beyond $n \approx 15$ the condition number exceeds machine precision, and the error plateaus at $\sim$1% regardless of n. The monomial Rayleigh-Ritz basis cannot improve further—this motivates finite elements with local, well-conditioned basis functions.

12 Conclusion

Aspect	Finding
Question 1 — Rayleigh-Ritz for the 1-D bar	
Part (a)	$u = a_2 x^2$ yields $a_2 = \frac{15P}{16EAL}$. Displacement underestimated by 37.5% at $x = L$; stress zero at the wall (exact: $2P/A$).
Part (b)	$u = a_2 x^2 + a_3 x^3$ yields $a_2 = \frac{15P}{4EAL}$, $a_3 = -\frac{5P}{2EAL^2}$. Displacement error improves to 16.7%, but stresses remain poor near $x = 0$.
Completeness	Omitting the linear term $a_1 x$ destroys completeness: $\varepsilon(0) = 0$ for all approximations. This illustrates Cook's warning (Sec. 3.6) that the lowest-order admissible terms must be included.
Stiffness	Both approximations underestimate displacements everywhere, confirming the Rayleigh-Ritz method produces solutions that are too stiff.
Question 2 — Bilinear rectangular element	
Shape functions	Starting from the bilinear field $u_i = p_r\, \alpha_{ir}$ and inverting the 4×4 nodal coefficient matrix $[A]$, the shape functions $N_I = \frac{1}{4ab}(a + \xi_I s)(b + \eta_I t)$ were derived. They satisfy $N_I(s_J, t_J) = \delta_{IJ}$ and $\sum_I N_I = 1$.
Interpolation	$u_i = N_I\, d_{iI}$ provides the displacement field in terms of nodal DOFs, extending the 1-D procedure (Cook, Sec. 3.8) to two dimensions via tensor-product construction.
Element eqs.	The element stiffness $k_{iI,kJ} = \int C_{ijkl}\, N_{I,j}\, N_{J,l}\, d\Omega$ was derived from the potential energy functional in full index notation.
Natural coords.	The same shape functions were re-derived using isoparametric coordinates $(\zeta, \eta) \in [-1, 1]^2$ with Jacobian $\det[\mathbf{J}] = ab$, confirming equivalence and enabling Gauss quadrature.
Numerical Validation	
OpenSees / FemLab	Both FEM tools reproduce the exact piecewise-linear solution to machine precision ($\mathcal{O}(10^{-14})$), confirming the analytical results and quantifying the Rayleigh-Ritz approximation errors.
Q2 stiffness	FemLab's Q4 element stiffness matches the analytical 2×2 Gauss result to relative error $\sim 10^{-16}$. Rank = 5 with 3 rigid-body modes confirmed.
Large-scale	The $n \times n$ system was assembled without loops and solved up to $n = 10{,}000$ using vectorization and CUDA GPU offloading. Assembly scales as $O(n^2)$, GPU solve as $O(n^3)$. The Hilbert-like condition number $\kappa_2 \sim 10^{22}$ prevents convergence beyond $n \approx 15$—motivating local FEM bases over global polynomials.

Table 7: Summary of key findings.

A Source Code

All source code is available at `https://github.com/adzetto/femlab-python`.

A.1 Julia — Rayleigh-Ritz Convergence Study

Listing 1: rayleigh_ritz_convergence.jl

```
#=
CE 512 HW3 - Rayleigh-Ritz Convergence Study  (fully vectorized + optional CUDA)

General n-term polynomial:  u(x) = a_2*x^2 + a_3*x^3 + ... + a_{n+1}*x^{n+1}
(a_1*x omitted as required by the homework)

Stiffness matrix:  K[a,b] = i*j / (i+j-1),  i=a+1, j=b+1   <- Hilbert-like
Force vector:      F[b]   = (1/2)^(b+1) + 1

Assembly, evaluation, and error quadrature are **loop-free** (broadcasting +
matrix-vector products).  When CUDA.jl is available the stiffness solve and the
dense evaluation are offloaded to the GPU for very large n.

Outputs .dat files for pgfplots import in LaTeX.
Normalized: P=1, EA=1, L=1.
=#

using LinearAlgebra, Printf

# -- Try to load CUDA ------------------------------------------------------
const HAS_CUDA = try
    @eval using CUDA
    CUDA.functional()
catch
    false
end
if HAS_CUDA
    println("CUDA available - GPU offloading enabled")
    print("Warming up CUDA JIT compilation... ")
    _k = CuArray(rand(5, 5))
    _f = CuArray(rand(5))
    _c = _k \ _f
    _ = Array(_c)
    println("done.")
end

const OUTDIR = @__DIR__

# -- Exact solution (normalized: P=EA=L=1) ---------------------------------
u_exact_v(xs) = ifelse.(xs .<= 0.5, 2.0 .* xs, 0.5 .+ xs)          # vectorized
sigma_exact_v(xs) = ifelse.(xs .< 0.5, 2.0, ifelse.(xs .> 0.5, 1.0, 1.5))

# -- Vectorized assembly ---------------------------------------------------
"""
    solve_rr(n) -> (ks, a)

Assemble the n x n stiffness matrix K and force vector F **without loops**,
then solve Ka = F.  Uses GPU if available and n is large enough.
"""
function solve_rr(n::Int)
    ks = collect(2:n+1)                       # exponents [2, 3, ..., n+1]
    iv = Float64.(ks)                         # row/col indices as floats

    # K[a,b] = i*j / (i+j-1) - outer product divided element-wise
    K = (iv * iv') ./ (iv .+ iv' .- 1.0)      # pure broadcasting, no loop

    # F[b] = 1/2^j + 1
    F = (1.0 ./ (2.0 .^ iv)) .+ 1.0           # pure broadcast

    # Solve - offload to GPU for large n
    if HAS_CUDA && n >= 256
        Kd = CuArray(K)
        Fd = CuArray(F)
```

```
        ad = Kd \ Fd
        a  = Array(ad)
    else
        a = K \ F
    end

    return ks, a
end

# -- Vectorized evaluation ----------------------------------------------------
"""
    eval_rr_vec(xs, ks, a) -> (u_vec, sigma_vec)

Evaluate displacement u(x) and stress sigma(x) at **all** points in `xs`
simultaneously using a single matrix-vector product (no loops).

    V[p, m] = xs[p]^ks[m]           ->  u  = V * a
    W[p, m] = ks[m] * xs[p]^(ks[m]-1)  ->  sigma = W * a
"""
function eval_rr_vec(xs::AbstractVector{Float64}, ks::Vector{Int}, a::Vector{Float64})
    # Vandermonde-like matrix: V[p,m] = x_p ^ k_m   (broadcasting, no loop)
    # log-exp trick avoids element-wise power loop:
    #   x^k = exp(k * log(x)),  with x=0 handled by replacing log(0)->-Inf
    logx = log.(max.(xs, 1e-300))              # column vector (npts,)
    kf   = Float64.(ks)'                       # row vector    (1, n)

    V = exp.(logx .* kf)                       # (npts, n) - all powers at once
    W = kf .* exp.(logx .* (kf .- 1.0))        # (npts, n) - derivative powers

    u_vec = V * a                              # matrix-vector, no loop
    sigma_vec = W * a
    return u_vec, sigma_vec
end

# ============================================================================
#  CONFIGURATION
# ============================================================================
x_fine   = collect(range(0.0, 1.0, length=500))
n_values = [1, 2, 3, 4, 5, 8, 12, 20, 30, 50]

# ============================================================================
#  DISPLACEMENT DATA
# ============================================================================
open(joinpath(OUTDIR, "data_displacement.dat"), "w") do io
    # header
    print(io, "x\texact")
    for n in n_values; print(io, "\tn$n"); end
    println(io)

    ue = u_exact_v(x_fine)                              # exact (vectorized)
    sols = [solve_rr(n) for n in n_values]             # precompute all
    u_cols = [eval_rr_vec(x_fine, ks, a)[1] for (ks, a) in sols]

    # write - single pass
    for p in eachindex(x_fine)
        @printf(io, "%.6f\t%.6f", x_fine[p], ue[p])
        for col in u_cols
            @printf(io, "\t%.6f", col[p])
        end
        println(io)
    end
end

# ============================================================================
#  STRESS DATA
# ============================================================================
open(joinpath(OUTDIR, "data_stress.dat"), "w") do io
    print(io, "x\texact")
    for n in n_values; print(io, "\tn$n"); end
    println(io)

    se = sigma_exact_v(x_fine)
    sols = [solve_rr(n) for n in n_values]
```

```
    sigma_cols = [eval_rr_vec(x_fine, ks, a)[2] for (ks, a) in sols]

    for p in eachindex(x_fine)
        @printf(io, "%.6f\t%.6f", x_fine[p], se[p])
        for col in sigma_cols
            @printf(io, "\t%.6f", col[p])
        end
        println(io)
    end
end

# ============================================================================
#  ERROR DATA  (vectorized quadrature - no inner loop over x)
# ============================================================================
n_range = 1:60
x_quad  = collect(range(0.0, 1.0, length=2000))
dx      = x_quad[2] - x_quad[1]
ue_quad = u_exact_v(x_quad)
se_quad = sigma_exact_v(x_quad)

open(joinpath(OUTDIR, "data_errors.dat"), "w") do io
    println(io, "n\tu_L2\tu_Linf\tsigma_L2\tsigma_Linf\tu_tip_pct")

    for n in n_range
        ks, a = solve_rr(n)
        u_rr, s_rr = eval_rr_vec(x_quad, ks, a)    # all 2000 pts, no loop

        du = abs.(u_rr .- ue_quad)
        ds = abs.(s_rr .- se_quad)

        u_L2   = sqrt(sum(du .^ 2) * dx)            # vectorized norms
        u_Linf = maximum(du)
        s_L2   = sqrt(sum(ds .^ 2) * dx)
        s_Linf = maximum(ds)

        u_tip    = u_rr[end]                          # x=1 is last point
        tip_pct  = abs(u_tip - 1.5) / 1.5 * 100

        @printf(io, "%d\t%.8e\t%.8e\t%.8e\t%.8e\t%.8e\n",
                n, u_L2, u_Linf, s_L2, s_Linf, tip_pct)
    end
end

# ============================================================================
#  PRINT SUMMARY
# ============================================================================
println("=" ^ 65)
println("CE 512 HW3 - Rayleigh-Ritz Convergence (vectorized, $(HAS_CUDA ? "GPU" : "CPU")
    )")
println("=" ^ 65)
println()
@printf("%-5s  %10s  %10s  %10s\n", "n", "u(L)", "u_err(%)", "||u||_L2")
println("-" ^ 45)

for n in n_values
    ks, a = solve_rr(n)
    u_rr, _ = eval_rr_vec(x_quad, ks, a)

    du   = abs.(u_rr .- ue_quad)
    u_L2 = sqrt(sum(du .^ 2) * dx)
    u_tip = u_rr[end]

    @printf("%-5d  %10.6f  %10.4f  %10.6f\n",
            n, u_tip, abs(u_tip - 1.5) / 1.5 * 100, u_L2)
end

println()
println("Files written:")
println("  data_displacement.dat")
println("  data_stress.dat")
println("  data_errors.dat")
```

A.2 Julia — Large-Scale Scaling Study (n up to 10 000)

Listing 2: rayleigh_ritz_scaling.jl — multi-threaded BLAS assembly and solve

```
#=
CE 512 HW3 - Large-Scale Rayleigh-Ritz: n x n System Scaling
(fully vectorized + CUDA GPU offloading)

Assembly, solve, and evaluation are **loop-free**.
* K assembled via outer-product broadcasting: K = iv*iv' ./ (iv.+iv'.-1)
* u(x) evaluated as a matrix-vector product  V*c  (Vandermonde approach)
* GPU offloading via CUDA.jl for n >= 256

Outputs data_scaling.dat for pgfplots.
Normalized: P=1, EA=1, L=1.
=#

using LinearAlgebra, Printf

# -- Try to load CUDA ---------------------------------------------------------
const HAS_CUDA = try
    @eval using CUDA
    CUDA.functional()
catch
    false
end
if HAS_CUDA
    println("CUDA available - GPU offloading enabled")
    print("Warming up CUDA JIT compilation... ")
    _k = CuArray(rand(5, 5))
    _f = CuArray(rand(5))
    _c = _k \ _f
    _ = Array(_c)
    println("done.")
end

# Use all available BLAS threads on CPU
nt = Sys.CPU_THREADS
BLAS.set_num_threads(nt)
println("BLAS threads: $nt")

const OUTDIR = @__DIR__

# -- Exact solution (vectorized) ----------------------------------------------
u_exact_vec(xs) = ifelse.(xs .<= 0.5, 2.0 .* xs, 0.5 .+ xs)
sigma_exact_vec(xs) = ifelse.(xs .< 0.5, 2.0, ifelse.(xs .> 0.5, 1.0, 1.5))

# -- Vectorized assembly (no loops) -------------------------------------------
"""
    assemble_vec(n) -> (K, F)    [CPU arrays]

Build K and F using pure broadcasting - zero explicit loops.
"""
function assemble_vec(n::Int)
    iv = Float64.(2:n+1)                          # [2, 3, ..., n+1]
    K  = (iv * iv') ./ (iv .+ iv' .- 1.0)         # outer product + broadcast
    F  = (1.0 ./ (2.0 .^ iv)) .+ 1.0              # broadcast
    return K, F
end

# -- Vectorized solve (CPU or GPU) --------------------------------------------
"""
    solve_system(K, F, n) -> (c, t_solve)

Solve Kc = F.  Offloads to GPU when CUDA is available and n >= 256.
Returns coefficient vector on CPU.
"""
function solve_system(K::Matrix{Float64}, F::Vector{Float64}, n::Int)
    if HAS_CUDA && n >= 256
        # GPU path
        t = @elapsed begin
            Kd = CuArray(K)
            Fd = CuArray(F)
```

```
            cd = Kd \ Fd
            c  = Array(cd)
        end
        return c, t
    else
        # CPU path - Cholesky (SPD) -> fallback LU -> fallback NaN
        local c
        t = @elapsed begin
            try
                c = cholesky(Symmetric(K)) \ F
            catch
                try
                    c = lu(K) \ F
                catch
                    c = fill(NaN, n)
                end
            end
        end
        return c, t
    end
end

# -- Vectorized evaluation (Vandermonde, no loops) ----------------------------
"""
    eval_u_vec(xs, n, c) -> u_vec

Evaluate u(x) = Sum[ c[a]*x^(a+1) ] at all points using a matrix-vector product.
"""
function eval_u_vec(xs::AbstractVector{Float64}, n::Int, c::Vector{Float64})
    ks   = Float64.(2:n+1)'                        # (1, n) row vector
    logx = log.(max.(xs, 1e-300))                   # (npts,) - safe log
    V    = exp.(logx .* ks)                         # (npts, n) Vandermonde-like
    return V * c                                    # matrix-vector, no loop
end

"""
    eval_sigma_vec(xs, n, c) -> sigma_vec

Evaluate sigma(x) = du/dx = Sum[ c[a]*k*x^(k-1) ] at all points.
"""
function eval_sigma_vec(xs::AbstractVector{Float64}, n::Int, c::Vector{Float64})
    ks   = Float64.(2:n+1)'
    logx = log.(max.(xs, 1e-300))
    W    = ks .* exp.(logx .* (ks .- 1.0))          # derivative Vandermonde
    return W * c
end

# ==============================================================================
#  CONFIGURATION
# ==============================================================================
n_vals = [1, 2, 3, 4, 5, 7, 10, 15, 20, 25, 30, 40, 50,
          75, 100, 150, 200, 300, 500, 750, 1000,
          1500, 2000, 3000, 5000, 7500, 10000]

const COND_MAX_N  = 2000   # cond() via SVD up to this n
const ERROR_MAX_N = 500    # L2 error via quadrature up to this n

# Quadrature grid - allocated once
x_quad = collect(range(0.0, 1.0, length=2000))
dx_q   = x_quad[2] - x_quad[1]
ue_q   = u_exact_vec(x_quad)
se_q   = sigma_exact_vec(x_quad)

# ==============================================================================
#  MAIN LOOP
# ==============================================================================
println("=" ^ 70)
println("CE 512 HW3 - Large-Scale Rayleigh-Ritz Scaling (vectorized, $(HAS_CUDA ? "GPU" : "CPU"))")
println("=" ^ 70)
println()

open(joinpath(OUTDIR, "data_scaling.dat"), "w") do io
```

```
    println(io, "n\tasm_s\tsolve_s\tcond_K\tresid\tu_tip\tu_tip_err\tu_L2\tsigma_L2")

    for n in n_vals
        @printf("n = %5d ... ", n)

        # -- Assembly (vectorized, timed) --
        local K, F
        t_asm = @elapsed begin
            K, F = assemble_vec(n)
        end

        # -- Condition number --
        kn = n <= COND_MAX_N ? cond(Symmetric(K)) : NaN

        # -- Solve --
        c, t_solve = solve_system(K, F, n)

        # -- Residual (vectorized) --
        resid = norm(K * c .- F) / norm(F)

        # -- Tip displacement u(1) = sum(c) since 1^k = 1 --
        u_tip   = sum(c)                              # no eval needed at x=1
        tip_err = abs(u_tip - 1.5) / 1.5 * 100

        # -- L2 errors (vectorized, no inner loop) --
        if n <= ERROR_MAX_N && isfinite(u_tip) && abs(u_tip) < 1e10
            ur = eval_u_vec(x_quad, n, c)
            sr = eval_sigma_vec(x_quad, n, c)

            du   = abs.(ur .- ue_q)
            ds   = abs.(sr .- se_q)
            u_l2 = sqrt(sum(du .^ 2) * dx_q)
            s_l2 = sqrt(sum(ds .^ 2) * dx_q)
        else
            u_l2 = NaN
            s_l2 = NaN
        end

        @printf(io, "%d\t%.6e\t%.6e\t%.6e\t%.6e\t%.6e\t%.6e\t%.6e\t%.6e\n",
                n, t_asm, t_solve, kn, resid, u_tip, tip_err, u_l2, s_l2)

        @printf("asm=%.4fs  solve=%.4fs  cond=%.2e  resid=%.2e  u(L)=%.6f\n",
                t_asm, t_solve, kn, resid, u_tip)
    end
end

println()
println("File written: data_scaling.dat")
println("Done.")
```

A.3 Python — OpenSees Validation

Listing 3: validate_opensees.py

```
"""
CE 512 HW3 - OpenSees Validation for Question 1
1-D bar: fixed at x=0, loads P at x=L/2 and x=L.
Compare exact solution and Rayleigh-Ritz approximations with FEM.
"""

import openseespy.opensees as ops
import numpy as np

# ====================================================================
# Parameters (normalized: P=1, EA=1, L=1)
# ====================================================================
P = 1.0
E = 1.0
A = 1.0
L = 1.0

```

```python
# ====================================================================
# OpenSees model: 1D bar with fine mesh
# ====================================================================
ops.wipe()
ops.model('basic', '-ndm', 1, '-ndf', 1)

n_elem = 20  # fine mesh for accuracy
dx = L / n_elem
mid_node = n_elem // 2 + 1  # node at x = L/2

# Create nodes
for i in range(n_elem + 1):
    ops.node(i + 1, i * dx)

# Fix left end
ops.fix(1, 1)

# Define material and elements (Truss element)
ops.uniaxialMaterial('Elastic', 1, E)
for i in range(n_elem):
    ops.element('Truss', i + 1, i + 1, i + 2, A, 1)

# Apply loads
ops.timeSeries('Linear', 1)
ops.pattern('Plain', 1, 1)
ops.load(mid_node, P)            # P at x = L/2
ops.load(n_elem + 1, P)          # P at x = L

# Analysis
ops.system('BandSPD')
ops.numberer('RCM')
ops.constraints('Plain')
ops.integrator('LoadControl', 1.0)
ops.algorithm('Linear')
ops.analysis('Static')
ops.analyze(1)

# ====================================================================
# Extract results
# ====================================================================
print("=" * 70)
print("CE 512 HW3 - OpenSees Validation (1-D Bar)")
print("=" * 70)

# Displacement at all nodes
x_os = np.array([ops.nodeCoord(i + 1, 1) for i in range(n_elem + 1)])
u_os = np.array([ops.nodeDisp(i + 1, 1) for i in range(n_elem + 1)])

# Exact solution
def u_exact(x):
    return np.where(x <= L / 2, 2 * P * x / (E * A), P * (L / 2 + x) / (E * A))

# Rayleigh-Ritz Part (a): u = 15P/(16EAL) * x^2
def u_rr_a(x):
    a2 = 15 * P / (16 * E * A * L)
    return a2 * x**2

# Rayleigh-Ritz Part (b): u = 15P/(4EAL)*x^2 - 5P/(2EAL^2)*x^3
def u_rr_b(x):
    a2 = 15 * P / (4 * E * A * L)
    a3 = -5 * P / (2 * E * A * L**2)
    return a2 * x**2 + a3 * x**3

u_ex = u_exact(x_os)
u_a = u_rr_a(x_os)
u_b = u_rr_b(x_os)

# ====================================================================
# Displacement comparison at key points
# ====================================================================
key_x = [0.0, 0.25, 0.5, 0.75, 1.0]
print("\n--- Displacement Comparison (normalized: u * EA / (PL)) ---\n")
```

```python
print(f"{'x/L':>5s}  {'Exact':>8s}  {'OpenSees':>8s}  {'Part(a)':>8s}  {'Part(b)':>8s}  {'FEM err':>8s}")
print("-" * 60)

for xv in key_x:
    idx = int(round(xv * n_elem))
    ue = u_exact(np.array([xv]))[0]
    uo = u_os[idx]
    ua = u_rr_a(np.array([xv]))[0]
    ub = u_rr_b(np.array([xv]))[0]
    err_fem = abs(uo - ue) / ue * 100 if ue > 0 else 0
    print(f"{xv:5.2f}  {ue:8.4f}  {uo:8.4f}  {ua:8.4f}  {ub:8.4f}  {err_fem:7.3f}%")

# ==================================================================
# Stress comparison
# ==================================================================
print("\n--- Stress Comparison (normalized: sigma * A / P) ---\n")
print(f"{'x/L':>5s}  {'Exact':>8s}  {'OpenSees':>8s}  {'Part(a)':>8s}  {'Part(b)':>8s}")
print("-" * 50)

# Stress from Rayleigh-Ritz
def sigma_rr_a(x):
    return 15 * P * x / (8 * A * L)

def sigma_rr_b(x):
    return 15 * P * x * (L - x) / (2 * A * L**2)

# Stress from OpenSees (element forces)
for xv in key_x:
    idx = int(round(xv * n_elem))
    # Exact stress
    if xv < 0.5:
        se = 2 * P / A
    elif xv > 0.5:
        se = P / A
    else:
        se = 2 * P / A  # left limit

    # OpenSees stress: from element axial force / A
    if idx > 0:
        elem_id = idx  # element to the left of node
        force = ops.eleResponse(elem_id, 'axialForce')
        if force:
            so = abs(force[0]) / A
        else:
            # Try basic force
            so = abs(ops.eleForce(elem_id, 1)) / A
    else:
        # At x=0, use first element
        force = ops.eleResponse(1, 'axialForce')
        if force:
            so = abs(force[0]) / A
        else:
            so = abs(ops.eleForce(1, 1)) / A

    sa = sigma_rr_a(np.array([xv]))[0]
    sb = sigma_rr_b(np.array([xv]))[0]
    print(f"{xv:5.2f}  {se:8.4f}  {so:8.4f}  {sa:8.4f}  {sb:8.4f}")

# ==================================================================
# Key values for LaTeX table
# ==================================================================
print("\n--- Values for LaTeX booktabs table ---\n")

# Displacements at x=L/2 and x=L
for xv, label in [(0.5, "L/2"), (1.0, "L")]:
    idx = int(round(xv * n_elem))
    ue = u_exact(np.array([xv]))[0]
    uo = u_os[idx]
    ua = u_rr_a(np.array([xv]))[0]
    ub = u_rr_b(np.array([xv]))[0]
    print(f"u({label}): exact={ue:.6f}, OpenSees={uo:.6f}, RR(a)={ua:.6f}, RR(b)={ub:.6f}")
```

```python

# Check: OpenSees should match exact for this linear problem
max_err = np.max(np.abs(u_os - u_ex))
print(f"\nMax |u_FEM - u_exact| = {max_err:.2e}")
print("(Should be ~0 since FEM with linear elements is exact for piecewise-linear solutions)")

ops.wipe()
print("\nOpenSees validation complete.")
```